



Universidad Tecnológica de Ciudad Juárez



QualityOps AI: validación preventiva de la calidad de software con DevOps e IA

Estándares y Métricas para el Desarrollo de Software

Prof. Francisco Javier Elizondo Luna

Ivan Valle Cortes – Matrícula: **24110751**

8 de octubre de 2026 -- Grupo DSM51



Educación
Secretaría de Educación Pública



MÁS CHIHUAHUA
más de lo bueno
GOBIERNO DEL ESTADO

UTP
DIRECCIÓN GENERAL DE UNIVERSIDADES
TECNOLÓGICAS Y POLITÉCNICAS

UTCJ

I. INTRODUCCIÓN

Este reporte documenta **QualityOps AI**, el programa que desarrollé para la materia Estándares y Métricas para el Desarrollo de Software. Es una herramienta **DevOps con IA**: cada vez que alguien propone un cambio en un repositorio de GitHub, ejecuta las pruebas, calcula métricas de producto, de proceso y de proyecto, aplica un *quality gate* (una barrera que impide integrar código que no cumple los umbrales) y publica un informe de calidad interpretado por un modelo de lenguaje. La herramienta se analiza a sí misma y también puede analizar cualquier otro repositorio Python.

El resultado final es un programa de 1.073 KLOC con 58 pruebas automatizadas, cobertura de 88.79 % y un pipeline de GitHub Actions que corre en cada Pull Request. Los defectos, su tiempo de detección y su tiempo de reparación se obtienen del historial de Git, sin registro manual.

Cómo leer este reporte. El capítulo 3 reúne los **resultados finales** (la ejecución final del programa, del pipeline y del dashboard); el resto del documento se apoya en ellos. Los capítulos 4 a 11 responden los ocho puntos de calidad de la actividad, el 12 y el 13 las métricas con IA y la estimación, y el 14 explica **cómo aplicar la herramienta a cualquier repositorio**. Cada capítulo empieza con un recuadro «En resumen».

Regla de evidencia. Nada en este reporte es inventado. Cada figura indica su origen: *salida del programa* (captura de una ejecución real), *captura intermedia* (tomada durante el desarrollo, con su commit), *gráfica elaborada para el reporte* (dibujada con `docs/reporte/graficas.py` a partir de `reports/metrics.json`) o *diagrama de elaboración propia* (Graphviz). Todas se pueden reproducir con los comandos del capítulo 14.

II. Índice

I. INTRODUCCIÓN.....	I
1. DESCRIPCIÓN DEL PROYECTO Y JUSTIFICACIÓN DEVOPS.....	- 1 -
1.1 <i>Qué es y qué problema resuelve</i>	<i>- 1 -</i>
1.2 <i>Justificación como proyecto DevOps.....</i>	<i>- 2 -</i>
1.3 <i>Tecnologías y frameworks.....</i>	<i>- 2 -</i>
2. ARQUITECTURA Y FUNCIONAMIENTO.....	- 3 -
2.1 <i>Arquitectura de componentes.....</i>	<i>- 3 -</i>
2.2 <i>Archivos del repositorio y su función</i>	<i>- 4 -</i>
2.3 <i>Pipeline de integración continua.....</i>	<i>- 5 -</i>
3. RESULTADOS FINALES	- 6 -
3.1 <i>Resumen de métricas.....</i>	<i>- 8 -</i>
4. CALIDAD DE SOFTWARE Y GRADO DE CUMPLIMIENTO.....	- 8 -
4.1 <i>Definición y forma de medir</i>	<i>- 8 -</i>
4.2 <i>Resultado.....</i>	<i>- 9 -</i>
4.3 <i>Requisitos abiertos.....</i>	<i>- 10 -</i>
5. INFORME DE CALIDAD EN UN FORMATO.....	- 11 -
5.1 <i>Estructura del informe.....</i>	<i>- 11 -</i>
5.2 <i>Reglas del dictamen de liberación.....</i>	<i>- 11 -</i>
6. REQUISITOS EXPLÍCITOS	- 12 -
6.1 <i>Requisitos funcionales.....</i>	<i>- 12 -</i>
6.2 <i>Requisitos no funcionales</i>	<i>- 12 -</i>
6.3 <i>Especificación por módulo</i>	<i>- 13 -</i>
6.4 <i>Validación contra herramientas de referencia</i>	<i>- 20 -</i>
6.5 <i>Trazabilidad entre requisitos y pruebas.....</i>	<i>- 22 -</i>
7. REQUISITOS DEL USUARIO Y EXPECTATIVAS DEL NEGOCIO	- 22 -
7.1 <i>Usuarios e historias de usuario</i>	<i>- 23 -</i>
7.2 <i>Expectativas del negocio e indicadores.....</i>	<i>- 23 -</i>
8. ASEGURAMIENTO DE LA CALIDAD DE SOFTWARE	- 24 -
8.1 <i>Plan SQA según IEEE 730</i>	<i>- 24 -</i>
8.2 <i>Gestión de configuración</i>	<i>- 24 -</i>
8.3 <i>Revisiones y registro de defectos.....</i>	<i>- 25 -</i>
8.4 <i>Verificación y validación.....</i>	<i>- 25 -</i>
9. CARACTERÍSTICAS DE CALIDAD: IDONEIDAD FUNCIONAL Y FIABILIDAD-	27 -

9.1	<i>Idoneidad funcional</i>	- 27 -
9.2	<i>Fiabilidad</i>	- 27 -
10.	ENFOQUE PREVENTIVO	- 29 -
10.1	<i>Concepto y controles implementados</i>	- 29 -
10.2	<i>Demostración del bloqueo</i>	- 29 -
10.3	<i>Detección temprana de los defectos reales</i>	- 32 -
11.	CARÁCTER SISTEMÁTICO	- 33 -
11.1	<i>Kanban y ciclo de trabajo</i>	- 33 -
11.2	<i>Trazabilidad de extremo a extremo</i>	- 34 -
11.3	<i>Normas aplicadas</i>	- 35 -
11.4	<i>Uso de IA durante el desarrollo</i>	- 35 -
12.	MÉTRICAS CALCULADAS APLICANDO IA	- 36 -
12.1	<i>Cómo participa la IA</i>	- 36 -
12.2	<i>Métricas de producto</i>	- 36 -
12.3	<i>Métricas de proceso</i>	- 38 -
12.4	<i>Métricas de proyecto</i>	- 39 -
13.	TÉCNICAS DE ESTIMACIÓN	- 41 -
13.1	<i>Juicio de expertos y tres puntos</i>	- 41 -
13.2	<i>Puntos de función y estimación análoga</i>	- 41 -
13.3	<i>Comparación con las horas reales</i>	- 42 -
14.	IMPLEMENTACIÓN EN OTRO REPOSITORIO	- 43 -
14.1	<i>Requisitos y consideraciones</i>	- 43 -
14.2	<i>Uso local</i>	- 43 -
14.3	<i>Integración al CI del repositorio</i>	- 44 -
14.4	<i>La llave de IA</i>	- 44 -
14.5	<i>Demostración en un repositorio ajeno</i>	- 45 -
14.6	<i>Configurar los umbrales</i>	- 47 -
15.	REPRODUCCIÓN DE RESULTADOS Y SALIDAS POSIBLES	- 48 -
16.	AMENAZAS A LA VALIDEZ	- 49 -
III.	CONCLUSIÓN	VI
IV.	REFERENCIAS	VII

ÍNDICE DE FIGURAS

Figura 1. Arquitectura de componentes de QualityOps AI (diagrama de elaboración propia)	3
Figura 2. Pipeline CI y regla de protección de main (diagrama de elaboración propia)	5
Figura 3. Métricas, quality gate e informe de la versión final (salida del programa, evidencia E20b)	6
Figura 4. Resumen del pipeline en GitHub Actions: quality gate y dictamen (salida del programa, evidencia E20d)	7
Figura 5. Dashboard: pestaña Resumen y cumplimiento (salida del programa, evidencia E20g)	7
Figura 6. Estado de los requisitos por categoría (gráfica elaborada para el reporte)	9
Figura 7. Dashboard: pestaña Producto (salida del programa, evidencia E20h)	13
Figura 8. Pruebas y linter de la versión final (salida del programa, evidencia E20a)	14
Figura 9. Defectos reales minados del historial de Git (salida del programa, evidencia E20c)	14
Figura 10. Dashboard: pestaña Proceso (salida del programa, evidencia E20i)	15
Figura 11. Dashboard: pestaña Proyecto (salida del programa, evidencia E20j)	15
Figura 12. Dashboard: pestaña Estimación (salida del programa, evidencia E20k)	16
Figura 13. Informe de calidad en Actions: pruebas, criterios y métricas (salida del programa, evidencia E20e)	17
Figura 14. Informe de calidad en Actions: defectos, interpretación y verificaciones (salida del programa, evidencia E20f)	18
Figura 15. Dashboard: pestaña AI Advisor (salida del programa, evidencia E20l)	19
Figura 16. Dashboard: riesgos, acciones y verificaciones (salida del programa, evidencia E20m)	19
Figura 17. Complejidad de QualityOps comparada con radon (captura intermedia, evidencia E07)	20
Figura 18. Cobertura de QualityOps comparada con coverage report (captura intermedia, evidencia E06)	21
Figura 19. Cobertura de líneas por archivo (gráfica elaborada para el reporte)	25
Figura 20. Resultado de las ejecuciones del pipeline (gráfica elaborada para el reporte)	27
Figura 21. Tolerancia a fallos: el LLM no responde y el informe se genera por reglas (captura intermedia, evidencia E10m)	27
Figura 22. PR de demostración: quality gate BLOQUEADO por complejidad 11 (salida del programa, evidencia E17b)	29
Figura 23. GitHub impide el merge del PR bloqueado (salida del programa, evidencia E17c)	29
Figura 24. Informe del PR bloqueado: dictamen NO LIBERAR (salida del programa, evidencia E17d) ..	30
Figura 25. Bloqueo local con un umbral de demostración (captura intermedia, evidencia E08a_gate) ...	30
Figura 26. Tiempo de detección y de reparación de cada defecto (gráfica elaborada para el reporte)....	31
Figura 27. Tablero Kanban al cierre del desarrollo (salida del programa, evidencia E02d)	33
Figura 28. Distribución de la complejidad por función (gráfica elaborada para el reporte)	36
Figura 29. Densidad de defectos por archivo afectado y global (gráfica elaborada para el reporte)	37
Figura 30. Ciclo de vida de un defecto y dónde se mide cada métrica (diagrama de elaboración propia)	37
Figura 31. Horas estimadas y reales por módulo (gráfica elaborada para el reporte)	39
Figura 32. Estimación por técnica frente a horas reales (gráfica elaborada para el reporte)	41
Figura 33. Defecto 5: estimación «0 h» en un repositorio sin registro de tiempos (v1.0.0) (salida del programa, evidencia E18a)	44
Figura 34. python-slugify: quality gate BLOQUEADO por complejidad (salida del programa, evidencia E18b)	45
Figura 35. python-slugify en el dashboard de QualityOps (salida del programa, evidencia E18c)	45
Figura 36. Plantilla de workflow en el CI de otro repositorio (fork de python-slugify) (salida del programa, evidencia E19)	46

ÍNDICE DE TABLAS

Tabla 1. Ficha técnica del proyecto	1
Tabla 2. Prácticas DevOps implementadas	2
Tabla 3. Tecnologías utilizadas	2
Tabla 4. Módulos del producto y sus métricas finales (commit 748d818)	4
Tabla 5. Estructura del repositorio	4
Tabla 6. Todas las métricas de la ejecución final (commit 748d818)	8
Tabla 7. Matriz de cumplimiento (docs/MATRIZ_CUMPLIMIENTO.md)	9
Tabla 8. Las 12 secciones del informe de calidad	11
Tabla 9. Criterios del dictamen (función dictamen() de report.py)	11
Tabla 10. Requisitos funcionales	12
Tabla 11. Requisitos no funcionales	13
Tabla 12. Especificación de complejidad y tamaño	13
Tabla 13. Especificación de pruebas y cobertura	14
Tabla 14. Especificación de minería de defectos szz	14
Tabla 15. Especificación de densidad de defectos	15
Tabla 16. Especificación de proceso y proyecto	15
Tabla 17. Especificación de estimación	16
Tabla 18. Especificación de cli y metrics.json	16
Tabla 19. Especificación de quality gate	16
Tabla 20. Especificación de pipeline	17
Tabla 21. Especificación de informe de calidad	17
Tabla 22. Especificación de ai advisor	18
Tabla 23. Especificación de dashboard y cumplimiento	19
Tabla 24. Requisitos funcionales y pruebas automatizadas que los verifican	21
Tabla 25. Usuarios de la herramienta	22
Tabla 26. Expectativas, indicador, meta y valor real	23
Tabla 27. Actividades del plan SQA	23
Tabla 28. Pull Requests integrados a main	24
Tabla 29. Defectos de producto minados del historial (ejecución final)	24
Tabla 30. Pruebas automatizadas por archivo	25
Tabla 31. Medidas de idoneidad funcional	26
Tabla 32. Medidas de fiabilidad	26
Tabla 33. Controles preventivos y lo que evitan	28
Tabla 34. Commits por tipo (git log, sin merges)	33
Tabla 35. Ejemplos de trazabilidad requisito → evidencia	33
Tabla 36. Normas aplicadas y cómo	34
Tabla 37. Procesos de ISO/IEC/IEEE 12207 y su práctica en el proyecto	34
Tabla 38. Reparto de responsabilidades entre el código y la IA	35
Tabla 39. Métricas de proceso de la ejecución final	38
Tabla 40. Línea base de estimación por tarea (horas)	40
Tabla 41. Conteo de puntos de función IFPUG (data/estimacion.json)	40
Tabla 42. Qué necesita el repositorio que se va a analizar	42
Tabla 43. Dónde ver cada resultado	43
Tabla 44. Escenarios de prueba y salida esperada	47

1. DESCRIPCIÓN DEL PROYECTO Y JUSTIFICACIÓN DEVOPS

En resumen: QualityOps AI es un *quality gate* inteligente para repositorios Python: mide, valida y reporta la calidad de cada cambio antes de que se integre a la rama principal. Es un proyecto DevOps porque su producto es una etapa del pipeline de integración continua, no una aplicación de negocio, y porque se desarrolló con las mismas prácticas que automatiza.

1.1 Qué es y qué problema resuelve

En muchos equipos la calidad se revisa tarde: alguien corre las pruebas a mano antes de entregar, nadie mide la complejidad y los defectos se registran en una hoja que nadie actualiza. QualityOps AI automatiza esa revisión y la convierte en una condición para integrar:

- **Mide en cada cambio:** pruebas, cobertura, complejidad, defectos y métricas de proceso y proyecto.
- **Bloquea antes de integrar:** si la cobertura baja de 75 %, una función supera complejidad 10 o falla una prueba, GitHub no permite el merge.
- **Obtiene los defectos del historial de Git**, verificables por cualquiera con `git log`.
- **Explica los resultados en lenguaje natural** con un modelo de lenguaje, sin dejar que la IA calcule las cifras.
- **Se integra a cualquier repositorio Python** con un archivo de workflow (capítulo 14).

Tabla 1. Ficha técnica del proyecto

Campo	Valor
Repositorio	github.com/TaikoVC/qualityops-ai (público, un solo autor)
Lenguaje	Python 3.14 (compatible con 3.11 o superior)
Tamaño	1.073 KLOC: 1073 líneas de código en 15 archivos, 87 funciones
Pruebas	58 automatizadas (pytest)
Historial	32 commits y 16 Pull Requests integrados
Pipeline	GitHub Actions, job «calidad», obligatorio para integrar a main
IA	Gemini (gemini-3.5-flash-lite) con respaldo por reglas
Versión analizada	commit 748d818 (2026-10-08 20:47)

1.2 Justificación como proyecto DevOps

DevOps une desarrollo y operación mediante automatización, integración continua, medición y retroalimentación rápida (Forsgren et al., 2018). En una calculadora o un sistema de restaurante DevOps sería solo un medio; en QualityOps AI **la práctica DevOps es el producto**.

Tabla 2. Prácticas DevOps implementadas

Práctica	Implementación	Dónde se ve
Integración continua	<code>.github/workflows/ci.yml</code> en cada push y Pull Request a main	Cap. 3 (Actions)
Pipeline como código	El pipeline está versionado y se revisa por PR	Diagrama del pipeline (cap. 2)
Quality gates (shift-left)	<code>quality_gate.py</code> termina con código 1 si no se cumplen los umbrales	Cap. 10 (bloqueo real)
Protección de la rama principal	Ruleset <code>proteger-main</code> : PR y check «calidad» obligatorios	Cap. 10
Ramas cortas y merge commits	Una rama por tarea; 16 PRs	Cap. 8
Métricas DORA	MTTR (tiempo de restauración) desde el historial	Cap. 12
Retroalimentación	Resumen en Actions, artefacto descargable y dashboard	Cap. 3
Reproducibilidad	Dependencias con versión fija; Python fijado en CI	<code>requirements.txt</code>
Gestión de secretos	Llave de IA como secreto, nunca en el código	Cap. 14

1.3 Tecnologías y frameworks

Tabla 3. Tecnologías utilizadas

Tecnología	Versión	Uso
Python	3.14	Lenguaje del programa y de las pruebas
pytest / pytest-cov	9.1.1 / 7.1.0	Pruebas y cobertura de líneas
radon	6.0.1	Complejidad ciclomática y conteo de líneas
ruff	0.16.10	Linter en local y en CI
Streamlit	1.65.0	Framework del dashboard web
Git, GitHub, GitHub Actions, GitHub Projects	—	Versiones, PRs, ruleset, CI y Kanban
Gemini API (Google AI Studio)	gemini-3.5-flash-lite	Interpretación en lenguaje natural
Graphviz y matplotlib	—	Solo para diagramas y gráficas de este reporte

2. ARQUITECTURA Y FUNCIONAMIENTO

En resumen: El programa tiene tres capas: un motor de métricas determinista (siete módulos que alimentan una CLI), el archivo `metrics.json` como único contrato entre capas, y una capa de validación y presentación (quality gate, informe, AI Advisor y dashboard). En GitHub Actions esas capas se ejecutan en siete pasos.

2.1 Arquitectura de componentes

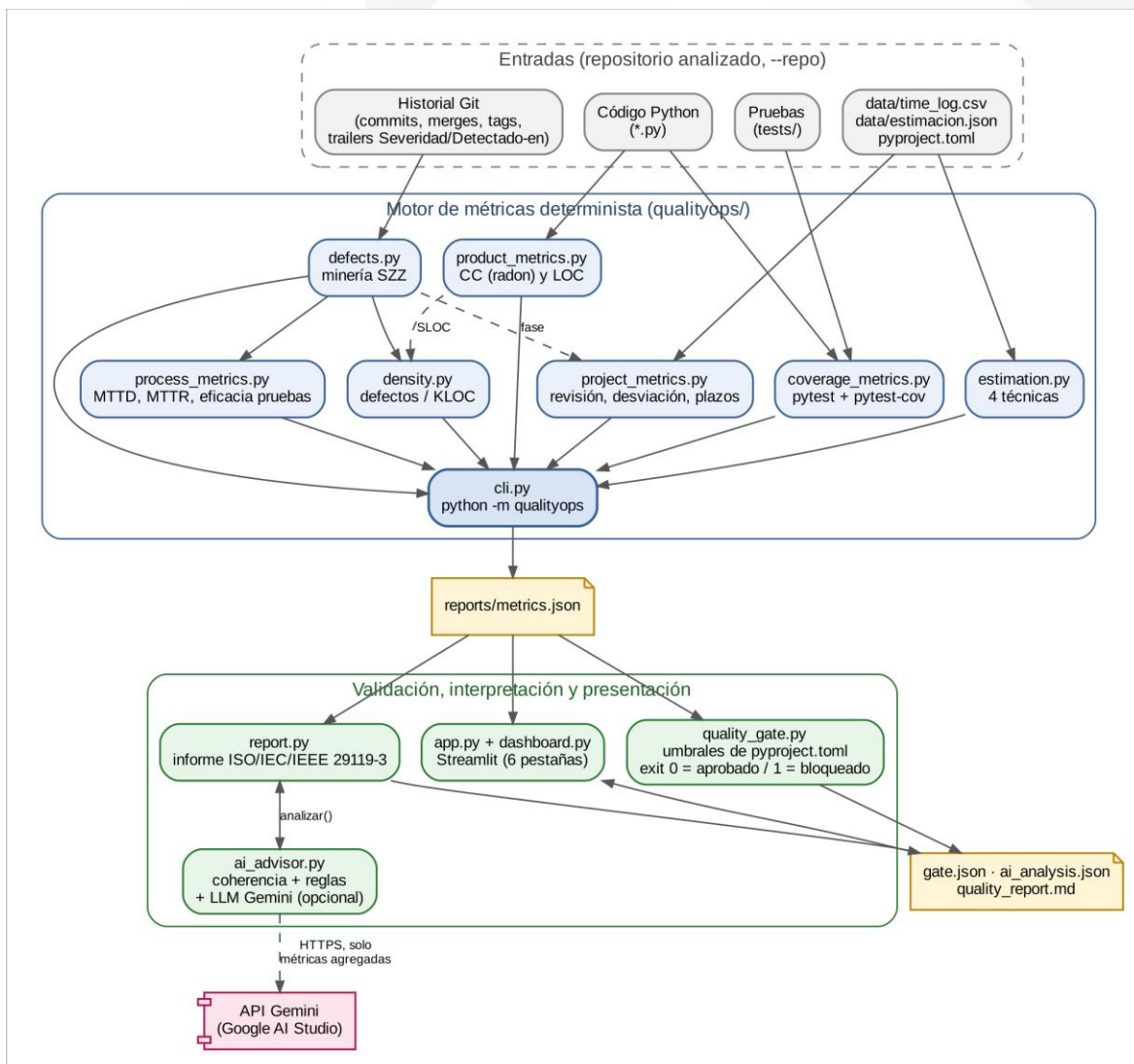


Figura 1. Arquitectura de componentes de QualityOps AI (diagrama de elaboración propia)

Origen y fin. Diagrama dibujado con Graphviz desde [docs/diagramas/arquitectura.dot](https://github.com/UTCJ/qualityops-ai/blob/main/docs/diagramas/arquitectura.dot); no es una salida del programa. Su fin es mostrar

de un vistazo qué archivo calcula cada métrica y cómo fluyen los datos hasta el informe y el dashboard.

Cómo leerlo. Las cajas azules son los archivos de `qualityops/` y las flechas indican quién usa a quién. `cli.py` reúne todo en `metrics.json` (amarillo); la capa verde valida, interpreta y presenta. La única conexión externa es la API de Gemini, que solo recibe métricas agregadas.

Cómo verificarlo. Cada flecha corresponde a un `import` real del código (por ejemplo, `cli.py` importa los siete módulos del motor) y se puede comprobar abriendo los archivos.

Tabla 4. Módulos del producto y sus métricas finales (commit 748d818)

Archivo	SLOC	Funciones	CC máx.	Cobertura %
<code>app.py</code>	76	0	—	85.29
<code>__init__.py</code>	1	0	—	100.0
<code>__main__.py</code>	3	0	—	0.0
<code>ai_advisor.py</code>	170	15	9	89.76
<code>cli.py</code>	73	5	2	100.0
<code>coverage_metrics.py</code>	91	6	7	80.6
<code>dashboard.py</code>	37	5	8	100.0
<code>defects.py</code>	163	15	9	83.87
<code>density.py</code>	37	4	4	61.54
<code>estimation.py</code>	55	7	9	100.0
<code>process_metrics.py</code>	27	3	7	100.0
<code>product_metrics.py</code>	95	7	6	80.33
<code>project_metrics.py</code>	61	8	7	92.73
<code>quality_gate.py</code>	69	5	6	97.83
<code>report.py</code>	115	7	9	97.37

2.2 Archivos del repositorio y su función

Tabla 5. Estructura del repositorio

Ruta	Función
<code>qualityops/</code>	Código de la herramienta: un archivo por responsabilidad (tabla anterior); se ejecuta con <code>python -m qualityops</code>
<code>app.py</code>	Dashboard Streamlit (6 pestañas) que lee <code>reports/</code>
<code>tests/</code>	58 pruebas automatizadas, un archivo por módulo
<code>.github/workflows/ci.yml</code>	Pipeline de integración continua (job «calidad»)
<code>pyproject.toml</code>	Configuración de ruff y pytest, umbrales del gate y calendario de sprints
<code>requirements.txt</code>	Dependencias con versión fija
<code>data/time_log.csv</code>	Estimación (O, M, P) y tiempo real de cada tarea
<code>data/estimacion.json</code>	Conteo de puntos de función y proyecto de referencia
<code>data/ci_runs.json</code>	Historial de ejecuciones del pipeline (<code>gh run list</code>)
<code>reports/</code>	Salidas generadas: <code>metrics.json</code> , <code>gate.json</code> , <code>ai_analysis.json</code> , <code>quality_report.md</code> (no se versionan)
<code>docs/GUIA_DE_USO.md</code>	Cómo aplicar la herramienta a cualquier repositorio
<code>docs/ejemplos/qualityops-ci.yml</code>	Plantilla de workflow para otros repositorios
<code>docs/MATRIZ_CUMPLIMIENTO.md</code>	Matriz viva de requisitos, evidencia y estado

Ruta	Función
docs/DECISIONES.md, docs/AI_LOG.md, docs/NORMAS.md, docs/EVIDENCIAS.md	Decisiones, uso de IA, normas aplicadas y registro de evidencias
docs/diagramas/, docs/reporte/	Fuentes de los diagramas, script de gráficas y este reporte
evidence/	Capturas con ID, descripción y fecha en el nombre

El código está documentado para su análisis: los 15 módulos tienen una cabecera que explica qué hacen y cómo se ejecutan, 71 de 90 funciones tienen docstring (las 19 restantes son auxiliares internas de pocas líneas) y las reglas que no son obvias llevan comentarios con su motivo o su decisión (por ejemplo, «D20»).

2.3 Pipeline de integración continua

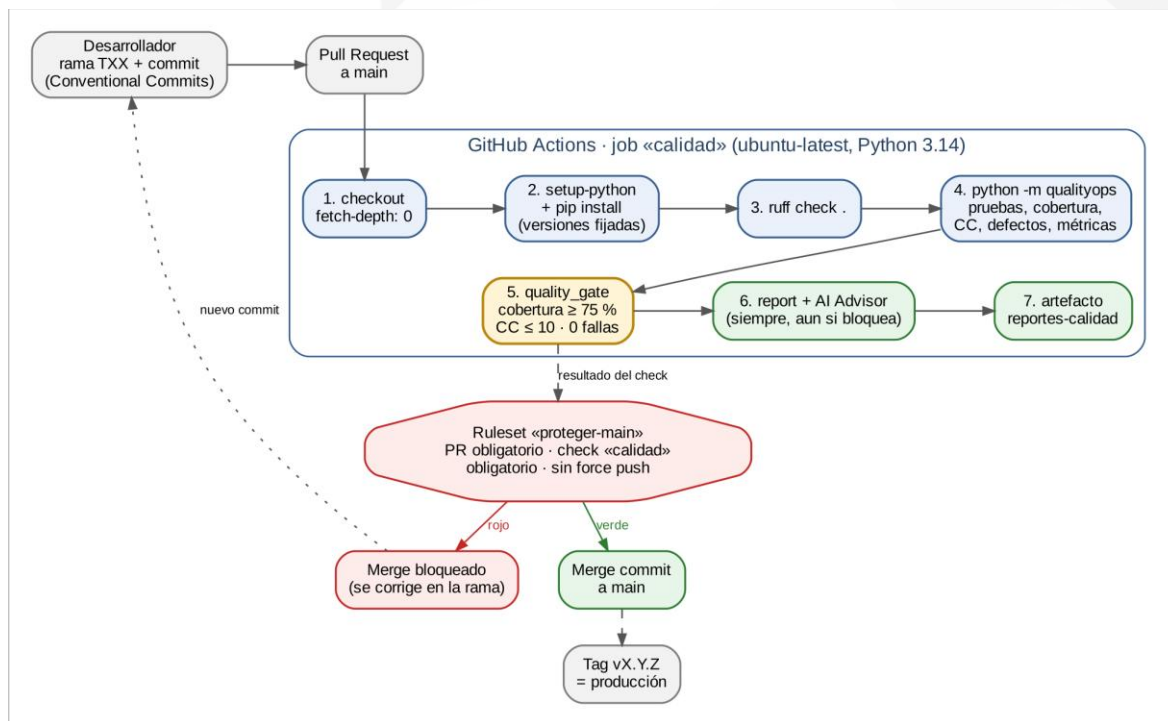


Figura 2. Pipeline CI y regla de protección de main (diagrama de elaboración propia)

Origen y fin. Diagrama dibujado con Graphviz desde [docs/diagramas/pipeline.dot](#); no es una salida del programa. Su fin es explicar qué ocurre desde que se abre un Pull Request hasta que el cambio se integra o se bloquea.

Cómo leerlo. Los pasos 1 a 7 son los del job «calidad». El paso 5 (quality gate) decide el color del check; el ruleset exige que esté en verde para hacer merge. El paso 6 se ejecuta siempre (`if: always()`) para que el informe explique por qué se bloqueó.

Cómo verificarlo. Los pasos coinciden uno a uno con [.github/workflows/ci.yml](#) y con el log de cualquier ejecución en la pestaña *Actions* del repositorio.

3. RESULTADOS FINALES

En resumen: Ejecución final sobre el commit 748d818: **58/58 pruebas**, cobertura **88.79 %**, complejidad máxima **9**, **4 defectos** de producto (3.73 def/KLOC), MTTR mediano **0.04 h**. El quality gate **APROBÓ** y el informe se generó con interpretación del modelo de lenguaje. Estas son las cifras de referencia de todo el reporte.

Las tres capturas siguientes resumen la ejecución final; las demás salidas finales (pruebas, defectos, cada pestaña del dashboard y cada sección del informe) aparecen en el capítulo 6, junto al módulo que las produce.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops --repo . --salida reports
QualityOps AI 0.1.0 - qualityops-ai @ 748d818
Pruebas:      58/58 aprobadas · 0 fallidas · 0 errores
Cobertura:    88.79 %
Complejidad:  promedio 3.77 · máximo 9
Tamaño:       1.073 KLOC en 15 archivos
Defectos:     4 · densidad 3.73 def/KLOC
MTTD / MTTR:  4.87 h / 0.06 h (n = 4 / 4)
Desviación:   -63.66 % (22 tareas terminadas)
Plazos:       27.27 % a tiempo
Estimación:   juicio 22.5 h · PERT 24.79 h · PF 4.24 h · análoga 4.79 h

Resultados guardados en reports\metrics.json
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.quality_gate --metrics reports/metrics.json

Quality gate - QualityOps AI
[OK ] Pruebas automatizadas sin fallas: 58/58 aprobadas (umbral total > 0 y 0 fallidas/errores)
[OK ] Cobertura de código: 88.79 (umbral >= 75.0 %)
[OK ] Complejidad ciclomática máxima: 9 (umbral <= 10)

Dictamen: APROBADO

(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.report
Informe generado: reports\quality_report.md · dictamen: LIBERAR CON CONDICIONES · interpretación: llm
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> |
```

Figura 3. Métricas, quality gate e informe de la versión final (salida del programa, evidencia E20b)

Qué muestra. `python -m qualityops` imprime el resumen de todas las métricas (pruebas, cobertura, complejidad, tamaño, defectos, MTTD/MTTR, desviación, plazos y las cuatro estimaciones); el gate evalúa sus tres criterios y `qualityops.report` genera el informe con interpretación `llm`.

Qué demuestra. El programa funciona de extremo a extremo y produce salida visible (PRG-06, PRG-07).

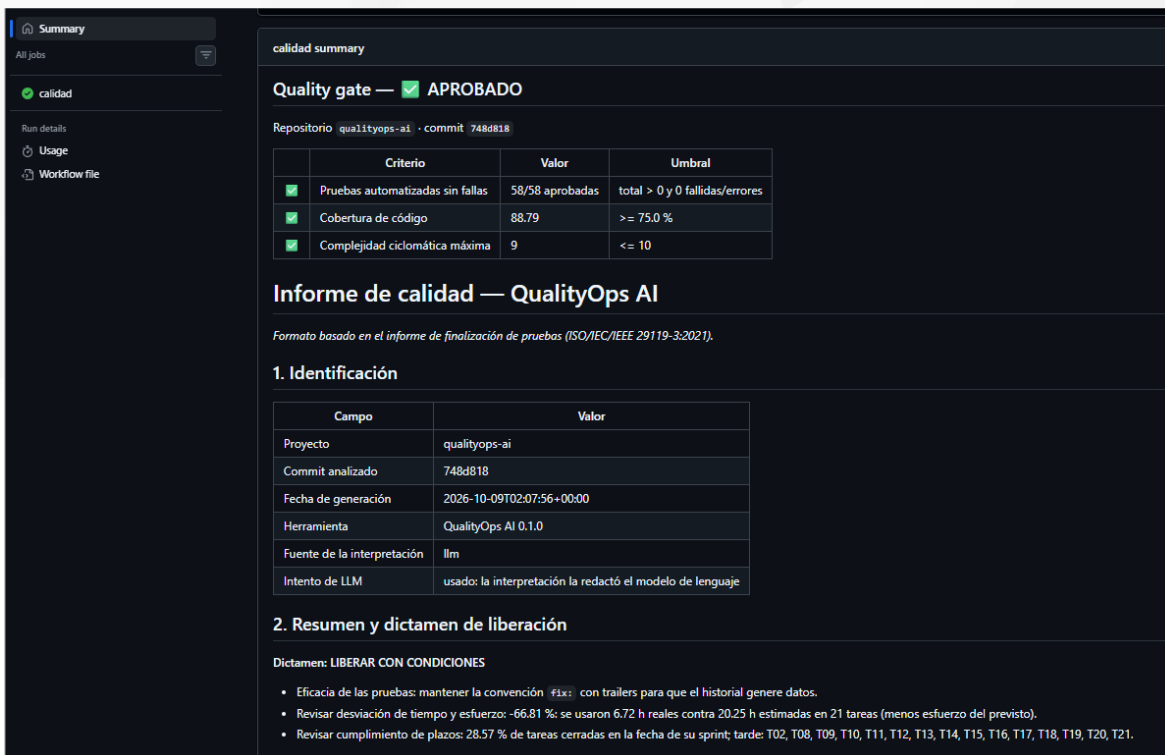


Figura 4. Resumen del pipeline en GitHub Actions: quality gate y dictamen (salida del programa, evidencia E20d)

Qué muestra. El *Summary* del job «calidad» con la tabla del quality gate y las secciones 1 y 2 del informe (commit analizado, fuente de la interpretación, intento de LLM y dictamen de liberación).

Qué demuestra. El mismo análisis corre en un servidor limpio de GitHub en cada cambio, con la IA configurada como secreto.

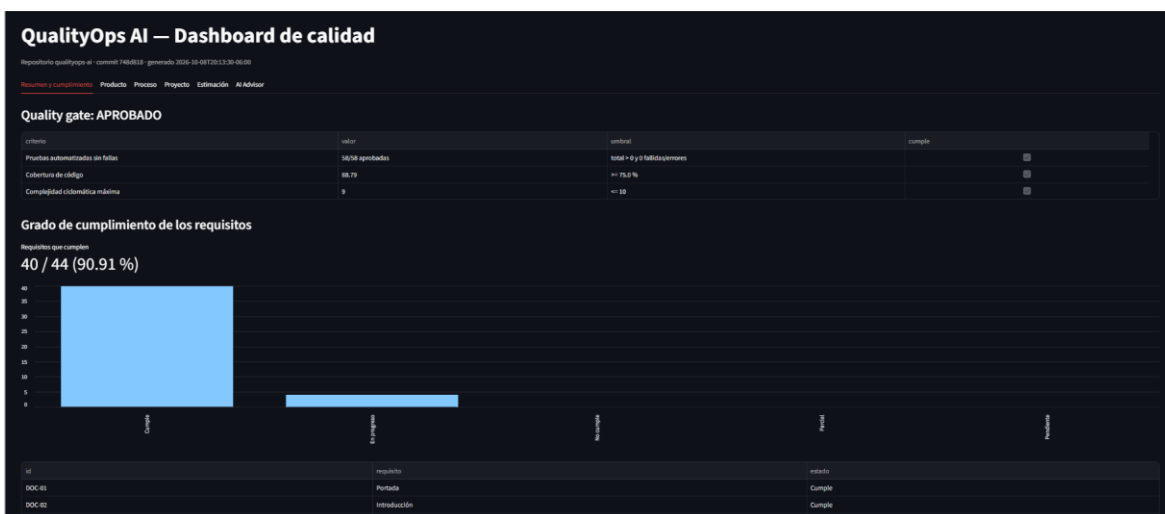


Figura 5. Dashboard: pestaña Resumen y cumplimiento (salida del programa, evidencia E20g)

Qué muestra. La tabla del quality gate y el grado de cumplimiento calculado desde la matriz de requisitos. Se tomó antes de cerrar CAL-05 y PRO-03, por eso indica 40 de 44; la matriz final está en el capítulo 4.

Qué demuestra. Los resultados finales se pueden explorar sin terminal (PRG-06, CAL-01).

3.1 Resumen de métricas

Tabla 6. Todas las métricas de la ejecución final (commit 748d818)

Grupo	Métrica	Valor	Referencia
Producto	Complejidad ciclomática	Promedio 3.77, máximo 9	≤ 10 (McCabe)
Producto	Cobertura de código	88.79 %	≥ 75 % (Google)
Producto	Densidad de defectos	3.73 def/KLOC (4 defectos)	1–25 (McConnell)
Proceso	MTTD	Mediana 0.24 h (n = 4)	Informativa
Proceso	MTTR	Mediana 0.04 h (n = 4)	< 1 h élite (DORA)
Proceso	Eficacia de las pruebas	Críticos: sin datos; todas: 75.0 %	≈ 85 % (Jones)
Proyecto	Eficacia de la revisión	50.0 %	Informativa
Proyecto	Desviación de tiempo y esfuerzo	-63.66 % (22 tareas)	Informativa
Proyecto	Cumplimiento de plazos	27.27 %	Informativa
Estimación	Juicio / PERT / PF / análoga	22.5 / 24.79 / 4.24 / 4.79 h	—
Calidad	Quality gate	APROBADO	3/3 criterios
Calidad	Grado de cumplimiento	42/44 (95.5 %)	Matriz viva

4. CALIDAD DE SOFTWARE Y GRADO DE CUMPLIMIENTO

En resumen: La calidad se mide contra 44 requisitos explícitos de la actividad, cada uno con evidencia. **42 de 44 (95.5 %) cumplen**; los 2 abiertos son PRG-09, MET-R3, explicados en la sección 4.3. Ninguno está en «No cumple».

4.1 Definición y forma de medir

ISO/IEC 25010:2023 define la calidad de un producto de software como el grado en que satisface las necesidades declaradas e implícitas de sus partes interesadas; IEEE 730-2014 agrega que la calidad se **asegura** con actividades planificadas. En el proyecto esto se traduce en una regla: un requisito solo cuenta como cumplido cuando existe evidencia que un tercero pueda revisar.

[docs/MATRIZ_CUMPLIMIENTO.md](#) lista cada requisito con su implementación, evidencia, criterio y estado (☒ Cumple, ☐ En progreso, ☐ Pendiente, ☒ No cumple). El

programa calcula el grado de cumplimiento con la función `resumen_matriz()` de `dashboard.py` y lo muestra en el dashboard:

Grado de cumplimiento (%) = requisitos en Cumple / total de requisitos × 100

4.2 Resultado

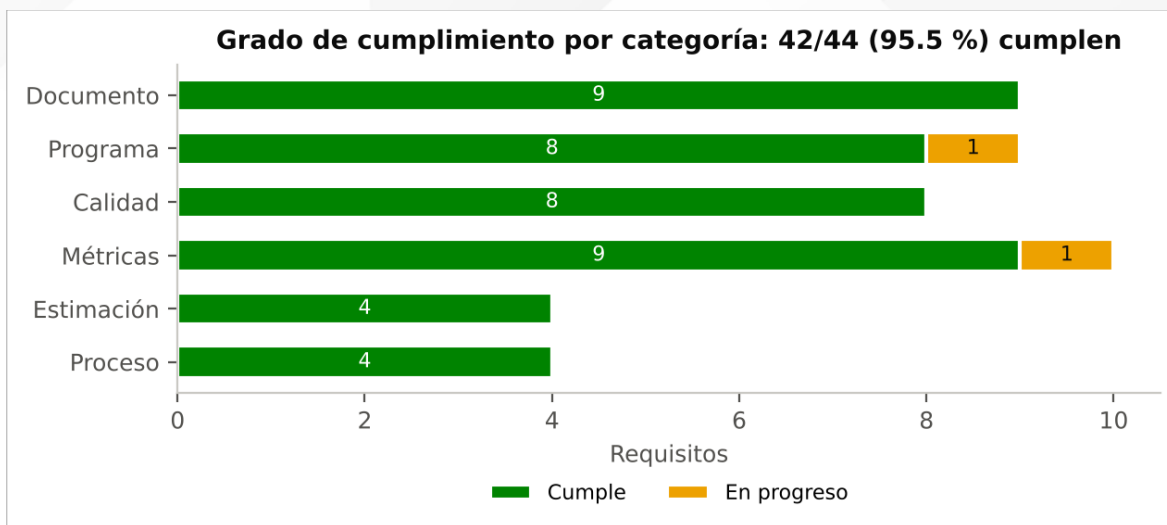


Figura 6. Estado de los requisitos por categoría (gráfica elaborada para el reporte)

Origen y fin. No es una pantalla del programa: la dibuja `docs/reporte/graficas.py` (función `cumplimiento_matriz`) con los datos de `reports/metrics.json` del commit 748d818. Su fin es ver en qué categorías de la actividad hay requisitos abiertos.

Cómo leerla. Cada barra es una categoría (Documento, Programa, Calidad, Métricas, Estimación, Proceso); el verde son requisitos con evidencia.

Cómo verificarla. El total de requisitos en Cumple es el mismo que muestra la pestaña «Resumen y cumplimiento» del dashboard (capítulo 3) y se puede contar a mano en la matriz.

Tabla 7. Matriz de cumplimiento (`docs/MATRIZ_CUMPLIMIENTO.md`)

ID	Requisito	Evidencia	Estado
DOC-01	Portada	Reporte final (E13): portada	Cumple
DOC-02	Introducción	Reporte final (E13): I. Introducción	Cumple
DOC-03	Índice tabulado	Reporte final (E13): II. Índice	Cumple
DOC-04	Imágenes explicadas	Reporte final (E13), E03	Cumple
DOC-05	Conclusión	Reporte final (E13): III. Conclusión	Cumple
DOC-06	≥ 10 hojas con tablas e imágenes	Reporte final (E13): cap. 6	Cumple
DOC-07	Plantilla UTCJ + referencias	Reporte final (E13): IV. Referencias (APA)	Cumple
DOC-08	Trabajo individual	E01; <code>git shortlog -sne</code> : Ivan Valle (commits) y TaikoVC (su cuenta de GitHub, merges)	Cumple
DOC-09	Normas aplicadas, claras y puntuales	Reporte final (E13): cap. 11.3	Cumple
PRG-01	Tecnologías DevOps	E08, E08b, E08e, PR #35	Cumple
PRG-02	IA en frameworks	E10h, E10i, E10k, PR #40, D19	Cumple

ID	Requisito	Evidencia	Estado
PRG-03	Automatización	E08, E08c, E08d	Cumple
PRG-04	Validación de software	E05, E05b, E10h, E10k	Cumple
PRG-05	GitHub + repositorio	E00, E00c, E01, README	Cumple
PRG-06	Ejecución y salida	E04b, E09a–h, E10h	Cumple
PRG-07	Métricas	E16a (todas las métricas en la ejecución final, commit 9d93529)	Cumple
PRG-08	Proyecto DevOps justificado	Reporte final (E13): cap. 1.3	Cumple
PRG-09	Revisión en clase	Reporte final (E13): cap. 14	En progreso (guion listo; ensayo pendiente)
CAL-01	Calidad y grado de cumplimiento	E09a, E09b	Cumple
CAL-02	Informe de calidad en formato	E10, E15b	Cumple
CAL-03	Requisitos explícitos	Reporte final (E13): cap. 6	Cumple
CAL-04	Requisitos de usuario y negocio	Reporte final (E13): cap. 7	Cumple
CAL-05	Aseguramiento de calidad	E20a, E20d (job «calidad» verde en 748d818 = tag v1.0.1)	Cumple
CAL-06	Idoneidad funcional y fiabilidad	Reporte final (E13): cap. 9, E16b	Cumple
CAL-07	Enfoque preventivo	E08a, E08b, E08g, E17a–E17d (PR #42 bloqueado por CC 11, cerrado sin merge)	Cumple
CAL-08	Carácter sistemático	E02, E02b, Reporte final (E13): cap. 11	Cumple
MET-00	Métricas aplicando IA	E10h, E10i	Cumple
MET-P1	Complejidad ciclomática	E07, 19d18b6, PR #28	Cumple
MET-P2	Cobertura	E06, 6ad52ac, PR #29	Cumple
MET-P3	Densidad de defectos	E11b, E11c, E11f, E10h	Cumple
MET-R1	MTTD	E11f, E10h	Cumple
MET-R2	MTTR	E11f, E10h	Cumple
MET-R3	Eficacia de pruebas	E11f	En progreso (0 críticos)
MET-J1	Eficacia de revisión	E11f, E15, E10g	Cumple
MET-J2	Desviación tiempo/esfuerzo por módulo	E11d	Cumple
MET-J3	Plazos (+ satisfacción opcional)	E11d	Cumple
EST-1	Juicio de expertos	E12	Cumple
EST-2	Estimación análoga	E12	Cumple
EST-3	Tres puntos (PERT)	E12	Cumple
EST-4	Puntos de función	E12	Cumple
PRO-01	Kanban	E02, E02c (cierre S0), E02d (tablero final: 18 en Done)	Cumple
PRO-02	Commits por cambio	E00c, E02b, E15c; tabla de PRs (cap. 8.2)	Cumple
PRO-03	Evidencia por sprint	E00–E20i; E14 = E20g (matriz calculada por el dashboard)	Cumple
PRO-04	Bitácora de IA	docs/AI_LOG.md	Cumple

4.3 Requisitos abiertos

- **PRG-09** (Revisión en clase): En progreso (guion listo; ensayo pendiente).
- **MET-R3** (Eficacia de pruebas): En progreso (0 críticos).

MET-R3 (eficacia de las pruebas sobre errores críticos) se deja abierto a propósito: el historial no tiene ningún defecto crítico y no se inventará uno. La fórmula está implementada y probada; con datos reales el programa reporta «sin datos» de forma explícita.

5. INFORME DE CALIDAD EN UN FORMATO

En resumen: En cada ejecución, local o en CI, el programa genera `reports/quality_report.md`: un informe con **formato fijo de 12 secciones** basado en el informe de finalización de pruebas de ISO/IEC/IEEE 29119-3:2021, que termina en un **dictamen de liberación** decidido por reglas explícitas. En GitHub se publica en el resumen de cada ejecución (Figura 4 y sección 6.3.10).

5.1 Estructura del informe

Tabla 8. Las 12 secciones del informe de calidad

#	Sección	Contenido	Fuente del dato
1	Identificación	Proyecto, commit, fecha, versión, fuente de la interpretación e intento de LLM	metrics.json, ai_analysis.json
2	Resumen y dictamen	Dictamen de liberación y recomendaciones	gate.json + reglas
3	Resultados de las pruebas	Total, aprobadas, fallidas, errores, omitidas	pytest (JUnit XML)
4	Criterios de salida	Cada criterio del gate con valor, umbral y si cumple	gate.json
5	Métricas de producto	Complejidad, cobertura, tamaño, densidad	metrics.json
6	Métricas de proceso	MTTD, MTTR, eficacia de pruebas, con n	metrics.json
7	Métricas de proyecto	Eficacia de revisión, desviación, plazos	metrics.json
8	Estimación	Las 4 técnicas y las horas reales	metrics.json
9	Defectos por severidad y fase	Matriz severidad × fase de detección	Minería SZZ
10	Interpretación	Valoración por reglas + texto del LLM	ai_advisor.py
11	Verificaciones de coherencia	5 comprobaciones aritméticas	ai_advisor.py
12	Aprobación	Responsable y fecha	Manual

Frente a ISO/IEC/IEEE 29119-3 el informe conserva los elementos del *test completion report*: identificación, resumen, desviaciones respecto a lo planeado (criterios de salida), métricas de la prueba, riesgos residuales (interpretación y recomendaciones) y aprobación.

5.2 Reglas del dictamen de liberación

Tabla 9. Criterios del dictamen (función `dictamen()` de `report.py`)

Dictamen	Condición	Significado
NO LIBERAR	El quality gate no aprobó	El cambio no debe integrarse
LIBERAR CON CONDICIONES	Gate aprobado, pero alguna métrica «sin datos» o una verificación de coherencia falla	Puede integrarse; hay huecos que atender
LIBERAR	Gate aprobado, todas las métricas con datos y 5/5 verificaciones	Puede integrarse sin reservas

En la ejecución final el dictamen es **LIBERAR CON CONDICIONES** porque «Eficacia de las pruebas» no tiene datos: no hay defectos críticos, así que la eficacia sobre críticos no se puede calcular. El programa prefiere declarar el hueco a mostrar un 100 % engañoso. El dictamen NO LIBERAR se observa en el PR bloqueado (evidencia E17d, capítulo 10).

6. REQUISITOS EXPLÍCITOS

En resumen: El programa tiene **15 requisitos funcionales** y **10 no funcionales**, cada uno con prioridad, módulo responsable, criterio de aceptación verificable y evidencia. Este capítulo los especifica módulo por módulo (entradas, proceso, salidas y reglas) y señala la evidencia que prueba que se cumplen.

6.1 Requisitos funcionales

Un requisito funcional describe **qué hace** el programa. Todos derivan de la actividad y cada uno tiene un criterio que se comprueba con una salida real.

Tabla 10. Requisitos funcionales

ID	Requisito	Prioridad	Módulo	Criterio de aceptación	Evidencia
RF-01	Calcular la complejidad ciclomática de cada función, con promedio, mediana, máximo y conteo de funciones con CC ≤ 10	Alta	product_metrics.py	Coincide función por función con <code>radon cc -s</code>	E07
RF-02	Contar LOC, SLOC, comentarios, blancos y KLOC del código de producto (sin pruebas)	Alta	product_metrics.py	Coincide con <code>radon raw -s</code>	E11c
RF-03	Ejecutar las pruebas del repositorio y medir la cobertura de líneas global y por archivo	Alta	coverage_metrics.py	Coincide con <code>coverage report</code>	E06
RF-04	Identificar los defectos reales del historial de Git (commits <code>fix:</code>), su commit inductor (SZZ), severidad y fase	Alta	defects.py	Defectos de los trailers = defectos minados	E11f
RF-05	Calcular la densidad de defectos global y por archivo (defectos/KLOC)	Alta	density.py	Densidad = defectos / KLOC (verificación 3)	E11b
RF-06	Calcular MTTD, MTTR y eficacia de las pruebas (críticos y todas las severidades)	Alta	process_metrics.py	Valores con n explícito; «sin datos» si n = 0	E11f
RF-07	Calcular eficacia de la revisión, desviación de tiempo/esfuerzo por módulo y cumplimiento de plazos	Alta	project_metrics.py	Coincide con el cálculo manual sobre <code>time_log</code>	E11d
RF-08	Aplicar 4 técnicas de estimación: juicio de expertos, análoga, tres puntos y puntos de función	Alta	estimation.py	Las 4 técnicas sobre la misma línea base	E12
RF-09	Ofrecer una CLI única que analice cualquier repositorio (<code>--repo</code>) y guarde <code>metrics.json</code>	Alta	cli.py	JSON completo y resumen en terminal	E04b, E10h
RF-10	Aplicar un quality gate con umbrales configurables y código de salida 0/1	Alta	quality_gate.py	Bloquea con umbral de demostración; aprueba con el real	E08a
RF-11	Ejecutar el análisis completo en GitHub Actions en cada push y Pull Request a main	Alta	ci.yml	Job «calidad» visible y obligatorio	E08, E08b
RF-12	Generar el informe de calidad de 12 secciones con dictamen de liberación	Alta	report.py	12 secciones en local y en CI	E10, E10k
RF-13	Verificar la coherencia de las métricas e interpretarlas con LLM, con respaldo por reglas	Alta	ai_advisor.py	fuelle = llm cuando hay llave; reglas si falla	E10h, E10m
RF-14	Mostrar las métricas en un dashboard web	Media	app.py, dashboard.py	6 pestañas con datos de <code>metrics.json</code>	E09a–h
RF-15	Calcular el grado de cumplimiento desde la matriz de requisitos	Media	dashboard.py	% = Cumple / total	E09b

6.2 Requisitos no funcionales

Un requisito no funcional describe **cómo** debe comportarse el programa. Los de mantenibilidad (RNF-02) y verificabilidad (RNF-03) los hace cumplir el quality gate en cada Pull Request.

Tabla 11. Requisitos no funcionales

ID	Atributo	Requisito	Cómo se verifica
RNF-01	Reproducibilidad	Dependencias con versión fija; Python 3.14 en local y en CI	requirements.txt, E00b
RNF-02	Mantenibilidad	Ninguna función con complejidad ciclomática > 10	Gate: CC máx. ≤ 10
RNF-03	Verificabilidad	Cobertura de líneas ≥ 75 % y 0 pruebas fallidas	Gate
RNF-04	Tolerancia a fallos	Si el LLM no responde o no hay llave, el informe se genera por reglas	E10m, E10g, pruebas
RNF-05	Seguridad	La llave de API vive como secreto; al LLM solo se envían métricas agregadas	ci.yml, D19
RNF-06	Portabilidad	Analiza cualquier repositorio Python con Git mediante <code>--repo</code>	D13, pruebas con repos sintéticos
RNF-07	Eficiencia	El pipeline completo termina en menos de 1 minuto	E08 (34 s), E10 (45 s)
RNF-08	Trazabilidad	Conventional Commits + trailers en cada <code>fix:</code>	E15c
RNF-09	Usabilidad	Salidas e informe en español, con valores, umbrales y explicación	E10k, E09
RNF-10	Integridad de datos	5 verificaciones de coherencia entre métricas en cada ejecución	Sección 11 del informe

6.3 Especificación por módulo

Cada tabla especifica un módulo con sus **entradas**, **proceso**, **salidas** y **reglas**, y la última fila indica dónde se ve el resultado final. El orden sigue el flujo de los datos.

6.3.1 Complejidad y tamaño (RF-01, RF-02) — `product_metrics.py`

Tabla 12. Especificación de complejidad y tamaño

Elemento	Descripción
Entradas	Ruta del repositorio (<code>--repo</code>); se recorren los <code>*.py</code> sin <code>tests/</code> , <code>docs/</code> , <code>.venv/</code> , cachés ni <code>test_*.py</code> .
Proceso	radon calcula la complejidad ciclomática (1 + decisiones) de cada función y cuenta LOC, SLOC, comentarios y blancos.
Salidas	Lista de funciones (archivo, nombre, línea, CC, rango A–F) y estadísticas: n, promedio, mediana, máximo, CC ≤ 10.
Reglas	Sin código, los valores son <code>null</code> (no se inventa un 0).
Resultado final	Figura 3, validación contra radon en la sección 6.4, figura siguiente



Figura 7. Dashboard: pestaña Producto (salida del programa, evidencia E20h)

Qué muestra. Cobertura, complejidad promedio/máxima, tamaño y densidad, con la cobertura por archivo y la tabla de funciones más complejas.

Qué demuestra. Las métricas de producto finales se consultan por archivo y por función (RF-01, RF-02).

6.3.2 Pruebas y cobertura (RF-03) — coverage_metrics.py

Tabla 13. Especificación de pruebas y cobertura

Elemento	Descripción
Entradas	Ruta del repositorio.
Proceso	Ejecuta <code>python -m pytest --cov=. --cov-report=json --junitxml</code> en un proceso aparte y lee ambos reportes.
Salidas	Pruebas (total, aprobadas, fallidas, errores, omitidas) y cobertura global y por archivo.
Reglas	Solo cuenta archivos de producto; una prueba fallida se reporta, no detiene la herramienta.
Resultado final	Figura 3, validación contra coverage en la sección 6.4, figura siguiente

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m pytest -q; ruff check .
..... [ 96%]
58 passed in 28.64s
All checks passed!
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai>
```

Figura 8. Pruebas y linter de la versión final (salida del programa, evidencia E20a)

Qué muestra. `python -m pytest -q` con todas las pruebas aprobadas y `ruff check .` sin observaciones.

Qué demuestra. El código entregado pasa su propia verificación antes de medirse (RF-03).

6.3.3 Minería de defectos SZZ (RF-04) — defects.py

Tabla 14. Especificación de minería de defectos szz

Elemento	Descripción
Entradas	Historial de Git: commits, merges (<code>--first-parent</code>) y tags vX.Y.Z.
Proceso	Por cada commit <code>fix</code> : lee los trailers y, con <code>git blame --porcelain</code> sobre el padre, identifica el commit inductor (Śliwerski et al., 2005).
Salidas	Por defecto: fix, inductor, fechas, MTDD, merge, MTTR, severidad y fase; lista de fixes excluidos con su motivo.
Reglas	Fase por trailer <code>Detectado-en</code> ; si falta, se deduce (antes del merge = revisión; después de un tag = producción).
Resultado final	tabla de defectos del capítulo 8, figura siguiente

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.defects --repo .

Minería de defectos (SZZ) - commit 748d818
Commits sin merge: 35 · merges de PR: 18 · tags de versión: v1.0.0, v1.0.1
Defectos de producto: 4 · fixes excluidos: 1

df1e4f2 menor      produccion (trailer) MTDD 18.79 h · MTTR 0.02 h · inductor cf916cb
fix(estimation): sin time_log.csv el juicio y PERT se reportaban como 0 h en lugar de sin datos
a02edd4 menor      revision (trailer) MTDD 0.25 h · MTTR 0.03 h · inductor c8b2162
fix(report): el informe y el dashboard explican si el LLM se usó, falló o no está configurado
f82e42f mayor      pruebas (trailer) MTDD 0.23 h · MTTR 0.05 h · inductor c8b2162
fix(ai_advisor): elimina el proveedor GitHub Models, retirado por GitHub el 2026-07-30
a6139f9 menor      revision (trailer) MTDD 0.22 h · MTTR 0.13 h · inductor cf916cb
fix(ai_advisor): nombres de archivo con guion bajo se mostraban en negritas en el informe
[excluido] 71f84cb fix(tests): aislar test_main_escribe_archivos de la API real de Gemini (no modifica código de producto)

MTDD mediana: 0.24 h (n = 4)
```

Figura 9. Defectos reales minados del historial de Git (salida del programa, evidencia E20c)

Qué muestra. `python -m qualityops.defects --repo .`: cada defecto con su commit de corrección, severidad, fase, MTDD, MTTR y commit inductor.

Qué demuestra. Los defectos no se capturaron a mano: cualquiera puede reproducir esta lista (RF-04).

6.3.4 Densidad de defectos (RF-05) — density.py

Tabla 15. Especificación de densidad de defectos

Elemento	Descripción
Entradas	Defectos minados y SLOC por archivo.
Proceso	Asigna cada defecto a los archivos de producto que tocó su corrección; global = defectos únicos / KLOC.
Salidas	Densidad global y por archivo (defectos/KLOC).
Reglas	Con código y sin defectos = 0.0; sin código = null.
Resultado final	Figura 3, Figura 7

6.3.5 Proceso y proyecto (RF-06, RF-07) — process_metrics.py, project_metrics.py

Tabla 16. Especificación de proceso y proyecto

Elemento	Descripción
Entradas	Defectos minados, <code>data/time_log.csv</code> y el calendario de <code>pyproject.toml</code> .
Proceso	MTTD = fix – inductor; MTTR = merge – fix; eficacias = defectos en la fase / con fase conocida; desviación = (real – estimado) / estimado; plazos = cierre ≤ fecha del sprint.
Salidas	Promedio y mediana con n; porcentajes con numerador y denominador; desviación por módulo; tareas tarde.
Reglas	Solo tareas terminadas; con n = 0 el valor es null («sin datos»).
Resultado final	Figura 3, figuras siguientes

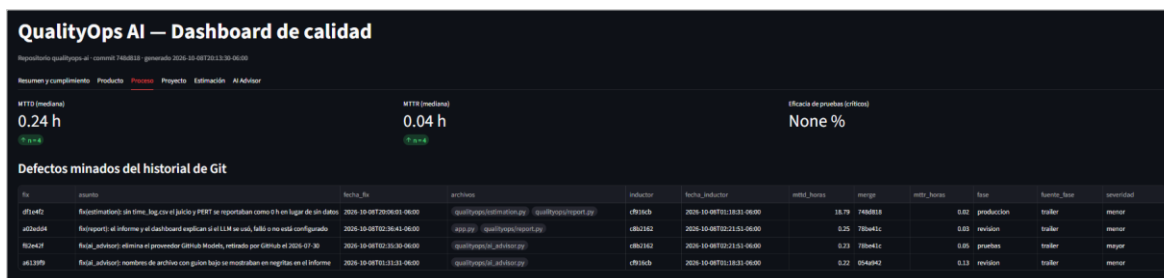


Figura 10. Dashboard: pestaña Proceso (salida del programa, evidencia E20i)

Qué muestra. MTTD, MTTR, eficacia de las pruebas y la tabla de defectos minados.

Qué demuestra. Las métricas de proceso salen del historial y se presentan con su n (RF-06).



Figura 11. Dashboard: pestaña Proyecto (salida del programa, evidencia E20j)

Qué muestra. Eficacia de la revisión, desviación por módulo y cumplimiento de plazos (capturada antes de registrar el tiempo de T26, por eso muestra 21 tareas; el valor final con 22 tareas está en la ejecución final del capítulo 3).

Qué demuestra. Las métricas de proyecto se calculan desde el registro de tiempos (RF-07).

6.3.6 Estimación (RF-08) — estimation.py

Tabla 17. Especificación de estimación

Elemento	Descripción
Entradas	Tareas de la línea base (<code>time_log.csv</code>) y <code>data/estimacion.json</code> .
Proceso	Juicio = $\sum M$; PERT = $\sum (O + 4M + P)/6$ y σ ; PF = $\sum \text{cantidad} \times \text{peso IFPUG}$; análoga = referencia $\times$ tamaño $\times$ factor.
Salidas	Horas por técnica, σ y rango del 95 %, PF y origen de la tasa.
Reglas	Sin tareas o sin datos, la técnica vale <code>null</code> (corrección del defecto 5).
Resultado final	Figura 3, figura siguiente

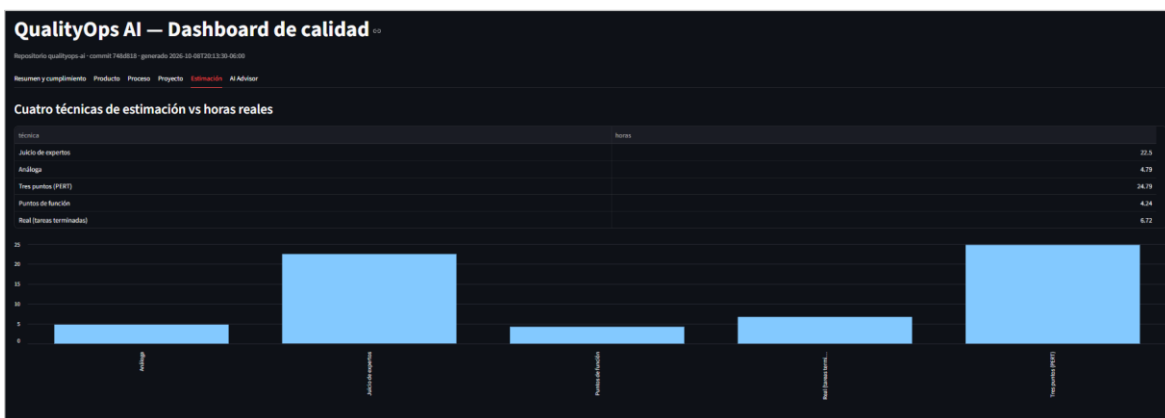


Figura 12. Dashboard: pestaña Estimación (salida del programa, evidencia E20k)

Qué muestra. Las cuatro técnicas frente a las horas reales registradas (capturada antes de registrar T26).

Qué demuestra. Las cuatro estimaciones las calcula el programa (RF-08).

6.3.7 CLI y metrics.json (RF-09) — cli.py

Tabla 18. Especificación de cli y metrics.json

Elemento	Descripción
Entradas	<code>--repo</code> (repositorio a analizar) y <code>--salida</code> (carpeta de reportes).
Proceso	Llama a todos los módulos y agrega herramienta, fecha, repositorio y commit analizado.
Salidas	<code>metrics.json</code> y resumen en terminal.
Reglas	El commit queda registrado para que cada resultado sea trazable.
Resultado final	Figura 3

6.3.8 Quality gate (RF-10) — quality_gate.py

Tabla 19. Especificación de quality gate

Elemento	Descripción
Entradas	<code>metrics.json</code> y <code>[tool.qualityops.umbrales]</code> de <code>pyproject.toml</code> (o <code>--config</code>).
Proceso	Tres criterios: pruebas sin fallas (total > 0), cobertura $\geq 75.0\%$, CC máxima ≤ 10 .
Salidas	Tabla en terminal, <code>gate.json</code> , tabla en el resumen de Actions; código de salida 0 o 1.
Reglas	Umbrales con referencia (Google Testing Blog; McCabe). Un valor faltante cuenta como falla.
Resultado final	Figura 3, Figura 4, bloqueos en el capítulo 10

6.3.9 Pipeline (RF-11) — .github/workflows/ci.yml

Tabla 20. Especificación de pipeline

Elemento	Descripción
Entradas	Evento push o pull_request hacia main.
Proceso	Siete pasos (diagrama del pipeline, capítulo 2).
Salidas	Check «calidad», resumen con gate e informe, artefacto <code>reportes-calidad</code> .
Reglas	El ruleset exige el check; <code>permissions: contents: read</code> .
Resultado final	Figura 4, PR #42 bloqueado en el capítulo 10

6.3.10 Informe de calidad (RF-12) — report.py

Tabla 21. Especificación de informe de calidad

Elemento	Descripción
Entradas	<code>metrics.json</code> , <code>gate.json</code> y el análisis del AI Advisor.
Proceso	Arma las 12 secciones y aplica las reglas del dictamen.
Salidas	<code>quality_report.md</code> y <code>ai_analysis.json</code> ; en CI se agrega al resumen.
Reglas	La fila «Intento de LLM» dice si la IA se usó, falló (y por qué) o no está configurada.
Resultado final	Figura 4, figura siguiente

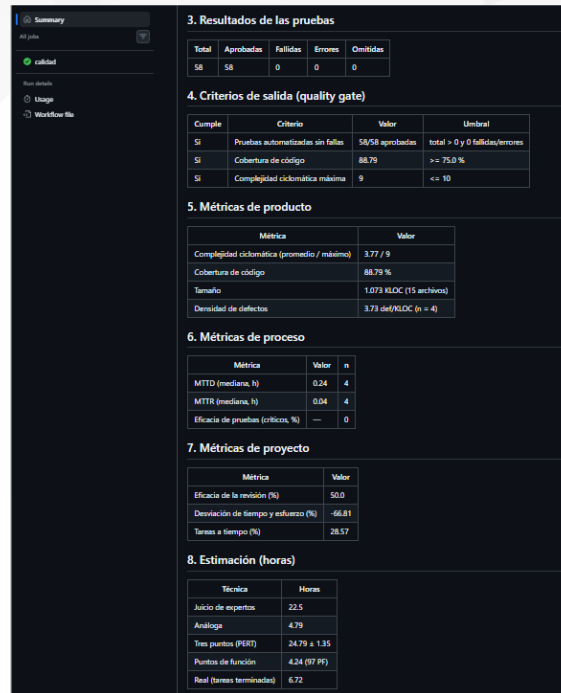


Figura 13. Informe de calidad en Actions: pruebas, criterios y métricas (salida del programa, evidencia E20e)

Qué muestra. Las secciones 3 a 8 del informe: pruebas, criterios de salida, métricas de producto, proceso y proyecto, y estimación.

Qué demuestra. El informe con formato fijo se publica donde lo ve cualquier revisor del PR (RF-12, CAL-02).

6.3.11 AI Advisor (RF-13) — ai_advisor.py

Tabla 22. Especificación de ai advisor

Elemento	Descripción
Entradas	metrics.json, gate.json y, opcional, la llave GEMINI_API_KEY.
Proceso	5 verificaciones de coherencia; interpretación por reglas; si hay llave, prompt fijo al LLM que prohíbe recalcular.
Salidas	Fuente (llm o reglas), interpretaciones, recomendaciones, verificaciones, texto del LLM o error_llm.
Reglas	429/503 se reintentan 3 veces; un tiempo agotado no se reintenta; cualquier error deja la ruta por reglas.
Resultado final	falla tolerada en el capítulo 9, figuras siguientes

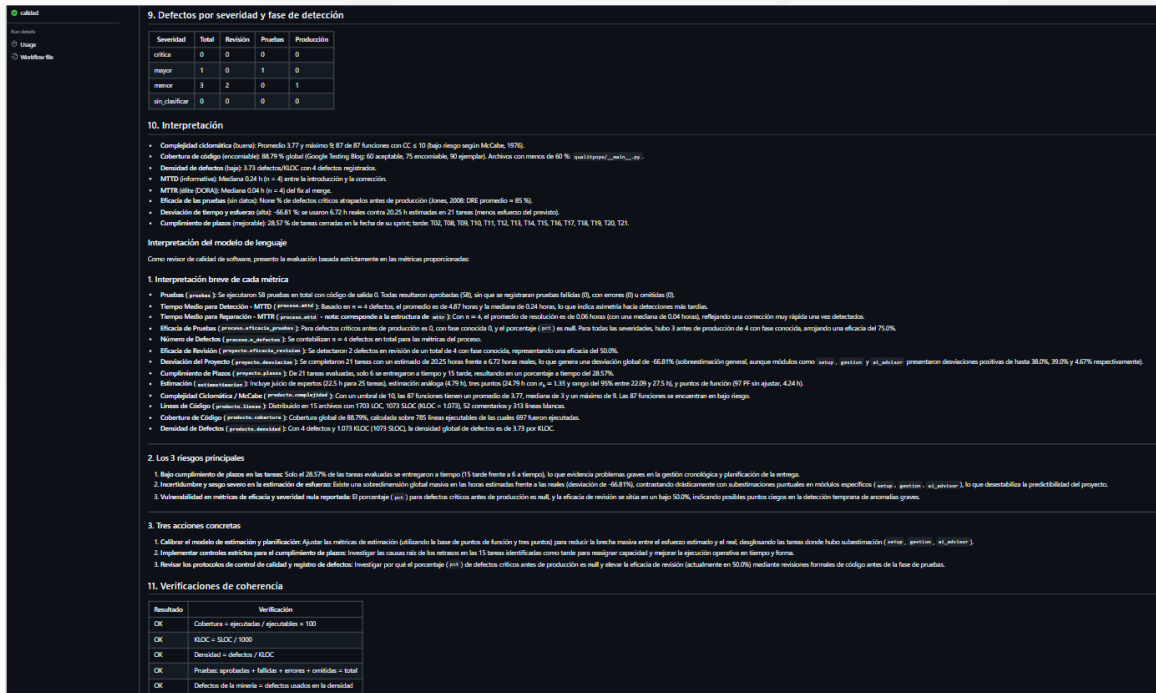


Figura 14. Informe de calidad en Actions: defectos, interpretación y verificaciones (salida del programa, evidencia E20f)

Qué muestra. Las secciones 9 a 11: defectos por severidad y fase, interpretación por reglas y del modelo de lenguaje, y las cinco verificaciones de coherencia.

Qué demuestra. La IA interpreta pero no altera las cifras; las verificaciones confirman su consistencia (RF-13).

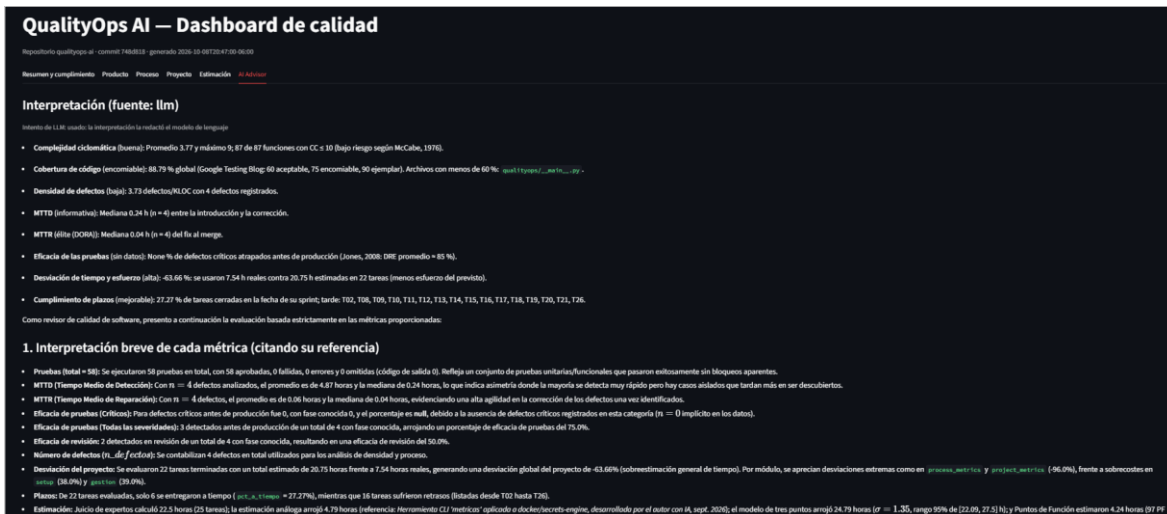


Figura 15. Dashboard: pestaña AI Advisor (salida del programa, evidencia E20l)

Qué muestra. «Interpretación (fuente: llm)», el estado «Intento de LLM: usado» y la interpretación de cada métrica escrita por el modelo.

Qué demuestra. La interpretación también es visible fuera del informe (RF-13).

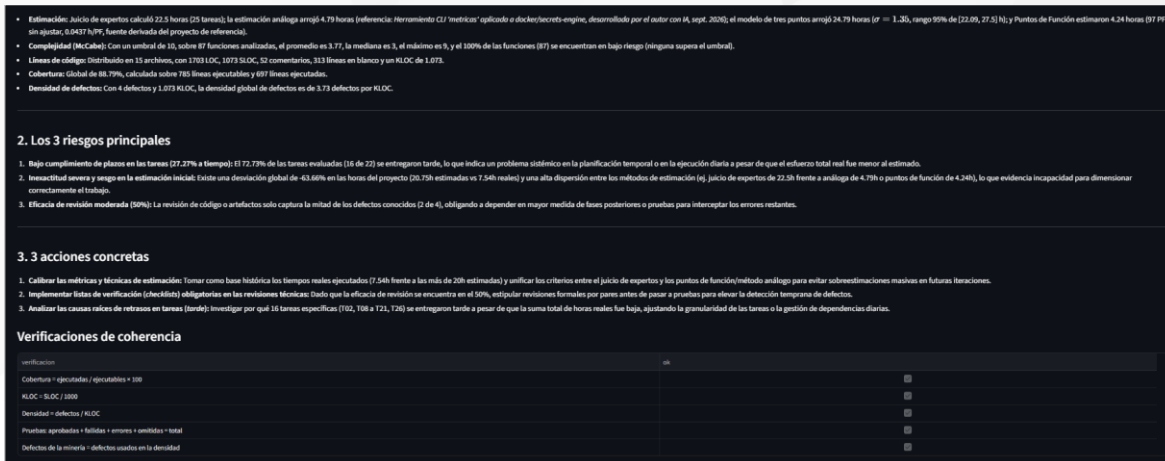


Figura 16. Dashboard: riesgos, acciones y verificaciones (salida del programa, evidencia E20m)

Qué muestra. Los 3 riesgos y las 3 acciones que propone el modelo de lenguaje y la tabla de las 5 verificaciones de coherencia, todas correctas.

Qué demuestra. La IA convierte las métricas en recomendaciones accionables sin alterar las cifras (MET-00).

6.3.12 Dashboard y cumplimiento (RF-14, RF-15) — app.py, dashboard.py

Tabla 23. Especificación de dashboard y cumplimiento

Elemento	Descripción
Entradas	reports/*.json y docs/MATRIZ_CUMPLIMIENTO.md (rutas configurables).
Proceso	Streamlit dibuja 6 pestañas; dashboard.py contiene la lógica sin interfaz para poder probarla.
Salidas	Resumen y cumplimiento, Producto, Proceso, Proyecto, Estimación, AI Advisor.
Reglas	Probado con streamlit.testing.AppTest (no se excluye de la cobertura).
Resultado final	Figura 5, pestañas del dashboard en esta sección

6.4 Validación contra herramientas de referencia

Para comprobar que los cálculos son correctos, comparé la salida de QualityOps con la de las herramientas estándar cuando cada módulo se integró. Estas capturas son de esa etapa; las cifras finales están en el capítulo 3.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.product_metrics --repo .

Complejidad ciclomática - C:\Users\LPC\proyectos\qualityops-ai
CC Rango Función (archivo:línea)
6 B analizar_complejidad (qualityops/product_metrics.py:68)
6 B contar_lineas (qualityops/product_metrics.py:117)
5 A encontrar_archivos_python (qualityops/product_metrics.py:45)
4 A _estadisticas (qualityops/product_metrics.py:100)
3 A es_archivo_de_prueba (qualityops/product_metrics.py:39)
2 A _imprimir (qualityops/product_metrics.py:149)
1 A _leer (qualityops/product_metrics.py:63)

Funciones: 7 · promedio: 3.86 · mediana: 4 · máximo: 6 · CC <= 10: 7
Tamaño: 2 archivos · LOC 175 · SLOC 96 · KLOC 0.096 · comentarios 7 · blancos 34

(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> radon cc -s qualityops
qualityops\product_metrics.py
F 68:0 analizar_complejidad - B (6)
F 117:0 contar_lineas - B (6)
F 45:0 encontrar_archivos_python - A (5)
F 100:0 _estadisticas - A (4)
F 39:0 es_archivo_de_prueba - A (3)
F 149:0 _imprimir - A (2)
F 63:0 _leer - A (1)
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> |
```

Figura 17. Complejidad de QualityOps comparada con radon (captura intermedia, evidencia E07)

Captura intermedia (T04, commit 19d18b6). No muestra los valores finales del proyecto (están en el capítulo 3); se conserva porque prueba un comportamiento que no se repite en la ejecución final.

Qué muestra. Arriba, `python -m qualityops.product_metrics --repo .` con las 7 funciones que existían; abajo, `radon cc -s qualityops`. Los 7 valores coinciden (6, 6, 5, 4, 3, 2, 1).

Qué demuestra. El cálculo de complejidad es correcto (criterio de RF-01).

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.coverage_metrics --repo .

Pruebas - C:\Users\LPC\proyectos\qualityops-ai
Total 8 · aprobadas 8 · fallidas 0 · errores 0 · omitidas 0 · código de salida 0

Ejecutables Ejecutadas % Archivo
1 1 100.0 qualityops\__init__.py
67 54 80.6 qualityops\coverage_metrics.py
61 49 80.33 qualityops\product_metrics.py

Cobertura global del código de producto: 80.62 % (104 de 129 líneas)

(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m pytest -q --cov=qualityops --cov-report=term
[100%]
===== tests coverage =====
----- coverage: platform win32, python 3.14.3-final-0 -----

Name Stmts Miss Cover
-----
qualityops\__init__.py 1 0 100%
qualityops\coverage_metrics.py 67 13 81%
qualityops\product_metrics.py 61 12 80%
TOTAL 129 25 81%
8 passed in 5.88s
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> |
```

Figura 18. Cobertura de QualityOps comparada con coverage report (captura intermedia, evidencia E06)

Captura intermedia (T05, commit 6ad52ac). No muestra los valores finales del proyecto (están en el capítulo 3); se conserva porque prueba un comportamiento que no se repite en la ejecución final.

Qué muestra. Arriba, QualityOps: 80.62 % (104 de 129 líneas); abajo, `pytest --cov --cov-report=term`: las mismas líneas por archivo (1/1, 67/54, 61/49), redondeadas a 81 %.

Qué demuestra. El cálculo de cobertura es correcto (criterio de RF-03).

6.5 Trazabilidad entre requisitos y pruebas

Tabla 24. Requisitos funcionales y pruebas automatizadas que los verifican

Requisitos	Archivo de pruebas	Pruebas
RF-01, RF-02	test_product_metrics.py	5
RF-03	test_coverage_metrics.py	3
RF-04	test_defects.py	5
RF-05	test_density.py	3
RF-06, RF-07	test_process_project_metrics.py	7
RF-08	test_estimation.py	7
RF-09	test_cli.py	1
RF-10	test_quality_gate.py	8
RF-12, RF-13	test_ai_advisor_report.py	14
RF-14, RF-15	test_dashboard.py	5
RF-11	Ejecución real del pipeline (no se prueba con pytest)	—
Total		58

Las pruebas de la minería crean repositorios Git sintéticos con fechas controladas; las del AI Advisor simulan la API de Gemini (respuesta correcta, error 503 con reintentos y tiempo agotado) para no depender de un servicio externo.

7. REQUISITOS DEL USUARIO Y EXPECTATIVAS DEL NEGOCIO

En resumen: Los usuarios son el desarrollador que propone cambios, el responsable de integrar a main, el evaluador que necesita evidencia y el equipo que adopta la herramienta en su repositorio. El negocio espera detectar problemas antes de integrar, con retroalimentación en menos de un minuto, sin costo y con datos imposibles de maquillar. Todas las expectativas medibles se cumplen salvo la de errores críticos, que no tiene datos.

7.1 Usuarios e historias de usuario

Tabla 25. Usuarios de la herramienta

Usuario	Necesidad	Cómo la atiende QualityOps AI
Desarrollador	Saber si su cambio rompe algo antes de pedir revisión	<code>python -m qualityops</code> en local o el check del PR
Responsable de integración	No integrar código que baje la calidad	Gate obligatorio por ruleset; dictamen de liberación
Evaluador / profesor	Verificar que las métricas son reales y reproducibles	Datos del historial de Git y comandos públicos
Equipo que adopta la herramienta	Integrarla sin reescribir su proyecto	<code>--repo</code> y plantilla de workflow (cap. 14)

- **HU-01.** Como desarrollador, quiero ver en el PR si pruebas, cobertura y complejidad cumplen los umbrales, para corregir antes de pedir revisión. *Criterio:* tabla del gate en el resumen del run.
- **HU-02.** Como responsable de integración, quiero que GitHub impida el merge si el gate falla. *Criterio:* PR bloqueado con el botón de merge deshabilitado (capítulo 10).
- **HU-03.** Como evaluador, quiero un informe con formato fijo y dictamen. *Criterio:* informe de 12 secciones en CI.
- **HU-04.** Como evaluador, quiero que los defectos salgan del historial. *Criterio:* `qualityops.defects` los reproduce.
- **HU-05.** Como equipo, quiero aplicar la herramienta a mi repositorio en minutos. *Criterio:* guía y plantilla probadas en un repositorio ajeno (capítulo 14).

7.2 Expectativas del negocio e indicadores

Tabla 26. Expectativas, indicador, meta y valor real

Expectativa	Indicador	Meta	Valor real	¿Cumple?
Detectar problemas antes de producción	Defectos antes de producción / con fase conocida	≥ 85 % (Jones, 2008)	75.0 % (3/4)	No
Retroalimentación rápida	Duración del pipeline por PR	< 1 min	34–59 s en las ejecuciones registradas	Sí
Reparar rápido	MTTR (mediana)	< 1 h (élite DORA)	0.04 h (n = 4)	Sí
Código mantenible	Funciones con CC ≤ 10	100 %	87/87	Sí
Código verificable	Cobertura de líneas	≥ 75 %	88.79 %	Sí
Datos confiables	Verificaciones de coherencia	5/5	5/5	Sí
Servicio disponible	Ejecuciones exitosas del pipeline	≥ 90 %	95.7 % (22/23)	Sí
Costo cero	Licencias y servicios	\$0	GitHub (repo público) y Gemini (nivel gratuito)	Sí
Contener errores críticos	Eficacia sobre críticos	≥ 85 %	Sin datos (0 críticos)	Sin datos

Requisitos implícitos que también se cuidan: no exponer secretos (la llave nunca se escribe en archivos), no fallar por causas externas (sin red o sin llave se usa la

ruta por reglas), mensajes claros (cada falla dice criterio, valor y umbral) y no inventar valores (con n = 0 se escribe «sin datos»).

8. ASEGURAMIENTO DE LA CALIDAD DE SOFTWARE

En resumen: Apliqué un plan SQA resumido según IEEE 730-2014: cada cambio pasa por linter, pruebas, métricas, quality gate, revisión del PR y CI obligatoria antes de integrarse; la configuración se controla con Git y los defectos se registran en el historial. Resultado: 16 PRs integrados, 58 pruebas, cobertura 88.79 % y 4 defectos de producto registrados.

8.1 Plan SQA según IEEE 730

Tabla 27. Actividades del plan SQA

Actividad	Cuándo	Herramienta	Criterio
Revisión automática de código	Cada commit y cada PR	ruff	0 errores
Pruebas unitarias e integración	Cada commit y cada PR	pytest	0 fallidas, 0 errores
Cobertura y complejidad	Cada PR	pytest-cov, radon	$\geq 75\%$, CC ≤ 10
Quality gate	Cada PR	quality_gate.py	Código 0
Revisión del PR con checklist	Antes de cada merge	Plantilla de PR	Checklist completo
Gestión de configuración	Siempre	Git, ramas, ruleset	Sin commits directos a main
Registro de defectos	En cada <code>fix</code> :	Trailers + defects.py	Severidad y fase en cada fix
Informe de calidad	Cada PR	report.py	12 secciones + dictamen
Registro de decisiones y uso de IA	En cada decisión	DECISIONES.md, AI_LOG.md	Motivo y alternativa

8.2 Gestión de configuración

Ningún cambio llegó a `main` sin Pull Request desde que se activó el ruleset. La tabla sale de `git log --merges --first-parent`.

Tabla 28. Pull Requests integrados a main

Merge	Fecha y hora	Pull Request y rama
9d93529	2026-10-08 08:37	PR #43 — rama T20-diagramas
ae04761	2026-10-08 08:04	PR #41 — rama T19b-registro
acd773f	2026-10-08 03:57	PR #40 — rama T17b-gemini
78be41c	2026-10-08 02:38	PR #39 — rama T18-T19-dashboard
054a942	2026-10-08 01:39	PR #38 — rama T12-T17-informe-ia
be6bb9c	2026-10-08 01:17	PR #37 — rama T15-estimacion
03ca184	2026-10-08 00:38	PR #36 — rama T13-T14-proceso-proyecto
00517a7	2026-10-08 00:17	PR #35 — rama T11-ci
ffb14d0	2026-10-06 12:49	PR #34 — rama T10-quality-gate
7f7f38e	2026-10-06 12:47	PR #33 — rama T09-pyproject
454864e	2026-10-06 12:31	PR #32 — rama T08-cli
2e9abd5	2026-10-05 21:35	PR #31 — rama T07-densidad
38ea0c8	2026-10-05 21:22	PR #30 — rama T06-defectos-szz
c7edea1	2026-10-05 10:44	PR #29 — rama T05-cobertura
8fa7528	2026-10-05 10:23	PR #28 — rama T04-complejidad

Merge	Fecha y hora	Pull Request y rama
3a8a2d5	2026-10-05 00:55	PR #26 — rama T02-kanban

8.3 Revisiones y registro de defectos

Cada PR se revisa con `docs/pull_request_template.md` (código comentado, ruff, pytest, prueba nueva cuando aplica, sin secretos, registro del uso de IA). Un defecto encontrado se corrige en el mismo PR con un commit `fix:` y trailers. Con esa regla, la revisión encontró **2 de 4** defectos (50.0 %).

Ejemplo real de commit de corrección con trailers (a6139f9)

`fix(ai_advisor):` nombres de archivo con guion bajo se mostraban en negritas en el informe

Severidad: menor

Detectado-en: revision

Evidencia: evidence/E15_defecto1_markdown_main_py_PR38_2026-10-08_0120.png

Refs: #17

Tabla 29. Defectos de producto minados del historial (ejecución final)

Fix	Descripción	Severidad	Fase	Inductor	MTTD h	MTTR h
a6139f9	nombres de archivo con guion bajo se mostraban en negritas en el informe	menor	revision	cf916cb	0.22	0.13
f82e42f	elimina el proveedor GitHub Models, retirado por GitHub el 2026-07-30	mayor	pruebas	c8b2162	0.23	0.05
a02edd4	el informe y el dashboard explican si el LLM se usó, falló o no está configurado	menor	revision	c8b2162	0.25	0.03
df1e4f2	sin time_log.csv el juicio y PERT se reportaban como 0 h en lugar de sin datos	menor	produccion	cf916cb	18.79	0.02

Además, la corrección `71f84cb` (`fix(tests):` aislar `test_main_escribe_archivos` de la API real de Gemini) se excluye de las métricas de producto porque no modifica código de producto: una prueba llamaba a la API real de Gemini y pytest tardaba 105 s. Es un defecto real del proceso de pruebas, no del producto.

8.4 Verificación y validación

La **verificación** comprueba que el programa calcula bien (pruebas con valores conocidos y comparación con radon y coverage, capítulo 6); la **validación** comprueba que resuelve el problema (el pipeline bloquea cambios malos, capítulo 10).

Tabla 30. Pruebas automatizadas por archivo

Archivo de pruebas	Pruebas
<code>test_ai_advisor_report.py</code>	14
<code>test_cli.py</code>	1
<code>test_coverage_metrics.py</code>	3
<code>test_dashboard.py</code>	5

Archivo de pruebas	Pruebas
test_defects.py	5
test_density.py	3
test_estimation.py	7
test_process_project_metrics.py	7
test_product_metrics.py	5
test_quality_gate.py	8
Total	58

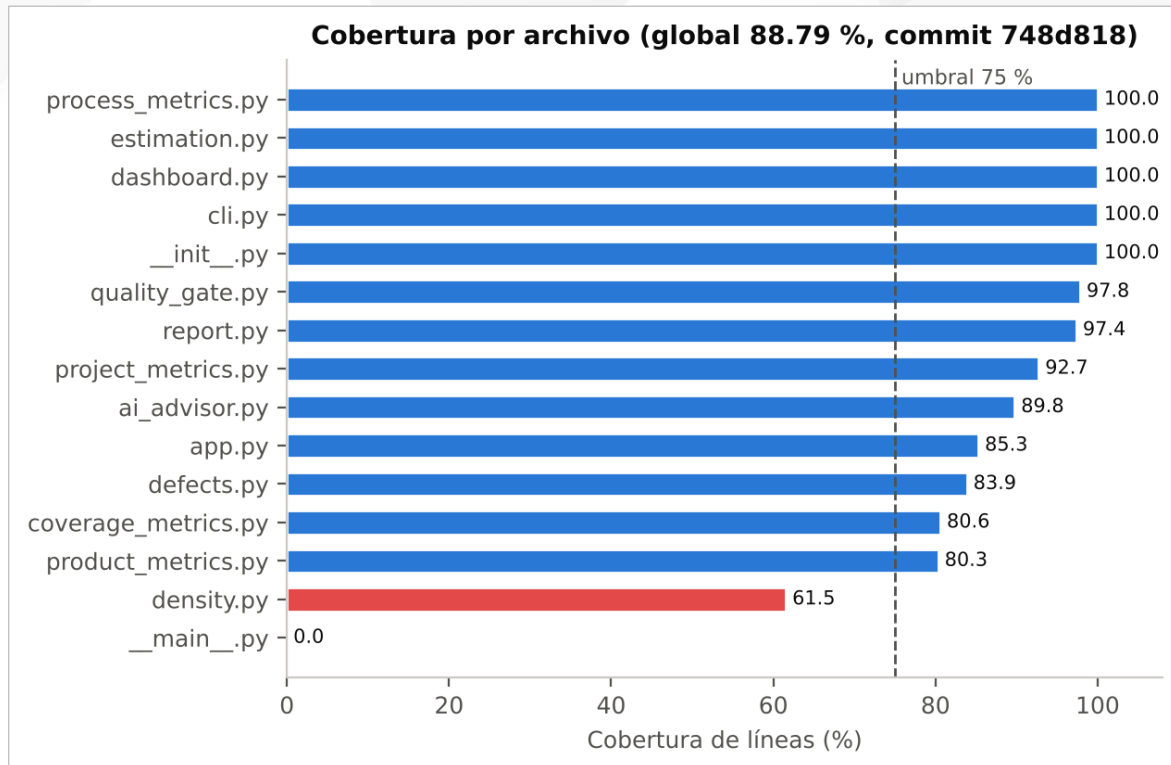


Figura 19. Cobertura de líneas por archivo (gráfica elaborada para el reporte)

Origen y fin. No es una pantalla del programa: la dibuja `docs/reporte/graficas.py` (función `cobertura_por_archivo`) con los datos de `reports/metrics.json` del commit 748d818. Su fin es mostrar en un solo vistazo qué archivos están menos probados respecto al umbral.

Cómo leerla. La línea punteada es el umbral del gate (75 %), que se aplica a la cobertura **global**. En rojo, los archivos por debajo: `density.py` (su función de línea de comandos no se prueba) y `__main__.py` (solo la llamada de arranque).

Cómo verificarla. Los porcentajes son los de `python -m qualityops.coverage_metrics --repo .` y los de la gráfica de la pestaña «Producto» del dashboard (capítulo 3).

9. CARACTERÍSTICAS DE CALIDAD: IDONEIDAD FUNCIONAL Y FIABILIDAD

En resumen: Con el modelo ISO/IEC 25010:2023 y medidas al estilo de ISO/IEC 25023: **idoneidad funcional** — 15/15 requisitos funcionales implementados, 58/58 pruebas correctas, ninguna función sin requisito. **Fiabilidad** — 4 defectos de producto corregidos, pipeline exitoso en 95.7 % (22 de 23 ejecuciones), tolerancia probada a 4 tipos de falla de la IA y recuperación mediana de 0.04 h.

9.1 Idoneidad funcional

Tabla 31. Medidas de idoneidad funcional

Subcaracterística	Medida	Valor	Evidencia
Compleitud funcional	RF implementados con evidencia / RF especificados	15/15 = 100 %	Cap. 6
Corrección funcional	Pruebas aprobadas / ejecutadas; valores iguales a la herramienta de referencia	58/58; radon 7/7; coverage igual	Cap. 3, E07, E06
Corrección (coherencia)	Verificaciones correctas / verificaciones	5/5	Sección 11 del informe
Pertinencia funcional	Módulos con requisito asociado / módulos funcionales	14/14 (0 funciones sin requisito)	Cap. 6

La corrección no depende solo de que las pruebas pasen: los valores se compararon contra radon y coverage, y en cada ejecución se comprueba la consistencia aritmética entre cobertura, KLOC, densidad y conteo de pruebas.

9.2 Fiabilidad

Tabla 32. Medidas de fiabilidad

Subcaracterística	Medida	Valor	Evidencia
Ausencia de fallos (madurez)	Defectos corregidos / detectados; densidad	4/4; 3.73 def/KLOC	E20c
Disponibilidad	Ejecuciones exitosas del pipeline / completadas	95.7 % (22 de 23 ejecuciones); la 1 falla corresponde al PR de demostración del bloqueo	data/ci_runs.json
Tolerancia a fallos	Fallas de la IA manejadas sin interrumpir el informe / probadas	4/4: sin llave, 503, tiempo agotado, modelo inexistente	E10m, pruebas
Recuperabilidad	MTTR = merge – fix	Mediana 0.04 h, promedio 0.06 h	E20c

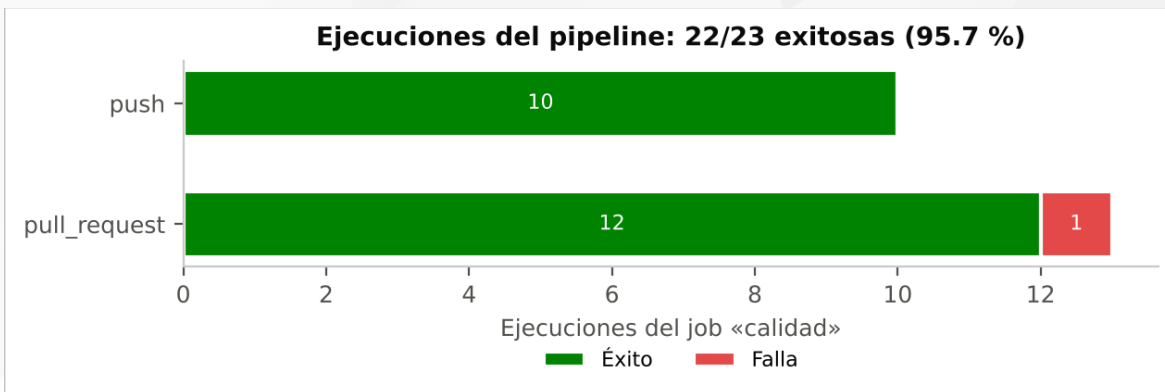


Figura 20. Resultado de las ejecuciones del pipeline (gráfica elaborada para el reporte)

Origen y fin. No es una pantalla del programa: la dibuja `docs/reporte/graficas.py` (función `ejecuciones_ci`) con los datos de `reports/metrics.json` del commit 748d818. Su fin es medir la disponibilidad del servicio de validación con datos de GitHub.

Cómo leerla. Cada barra es un tipo de evento (`pull_request` o `push`), dividido por resultado.

Cómo verificarla. Los datos vienen de `gh run list -L 100 --json ... > data/ci_runs.json` y coinciden con la pestaña *Actions* del repositorio.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m pytest -q
[100%]
35 passed in 105.37s (0:01:45)
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.report
Informe generado: reports\quality_report.md · dictamen: LIBERAR CON CONDICIONES · interpretación: reglas
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> Get-Content reports\ai_analysis.json -Encoding UTF8 | Select-String '"fuente"|"proveedor"|"error_llm"'
"fuente": "reglas",
"error_llm": "TimeoutError: The read operation timed out"
```

Figura 21. Tolerancia a fallos: el LLM no responde y el informe se genera por reglas (captura intermedia, evidencia E10m)

Captura intermedia (T17b, modelo gemini-3.8-flash). No muestra los valores finales del proyecto (están en el capítulo 3); se conserva porque prueba un comportamiento que no se repite en la ejecución final.

Qué muestra. La petición al modelo agotó el tiempo de espera; el informe se generó igual con `fuente: reglas` y el motivo en `error_llm`.

Qué demuestra. Una falla del servicio externo no rompe el pipeline ni oculta el problema (RNF-04).

10. ENFOQUE PREVENTIVO

En resumen: La prevención consiste en impedir que un defecto llegue a la rama principal en lugar de corregirlo después. En QualityOps AI es automática: el quality gate corre en cada Pull Request y el ruleset impide integrar si falla. Lo demostré bloqueando un cambio real con complejidad 11. De 4 defectos con fase conocida, 3 se detectaron antes de producción.

10.1 Concepto y controles implementados

ISO/IEC/IEEE 90003:2018 orienta la gestión de la calidad a la prevención: controlar el proceso para que el producto salga bien, en lugar de inspeccionar al final. En DevOps esto se llama *shift-left*: verificar lo más temprano posible, porque corregir un defecto cuesta más cuanto más tarde se detecta (Boehm, 1981).

Tabla 33. Controles preventivos y lo que evitan

Control	Cuándo actúa	Qué evita
Versiones fijas de dependencias	Instalación	Que un cambio externo rompa el build sin aviso
ruff (linter)	Antes de las pruebas	Errores de sintaxis, imports sin usar, malas prácticas
Pruebas automatizadas	Cada PR	Regresiones en funciones ya probadas
Umbral de complejidad ($CC \leq 10$)	Cada PR	Funciones difíciles de probar y mantener
Umbral de cobertura ($\geq 75\%$)	Cada PR	Código nuevo sin pruebas
Ruleset proteger-main	Al intentar el merge	Integrar con el check en rojo o sin PR
Revisión con checklist	Antes del merge	Defectos que las pruebas no ven
Verificaciones de coherencia	Cada informe	Publicar métricas contradictorias
Respaldo por reglas del AI Advisor	Cada informe	Quedarse sin informe si la IA falla
Secretos fuera del código	Siempre	Fuga de credenciales

10.2 Demostración del bloqueo

Abrí un Pull Request (#42) que agregaba [qualityops/demo_bloqueo.py](#): una función con una cadena de 10 `if/elif`, es decir, complejidad ciclomática 11. El archivo pasa el linter; solo el quality gate lo detiene. El PR se cerró sin integrarse.

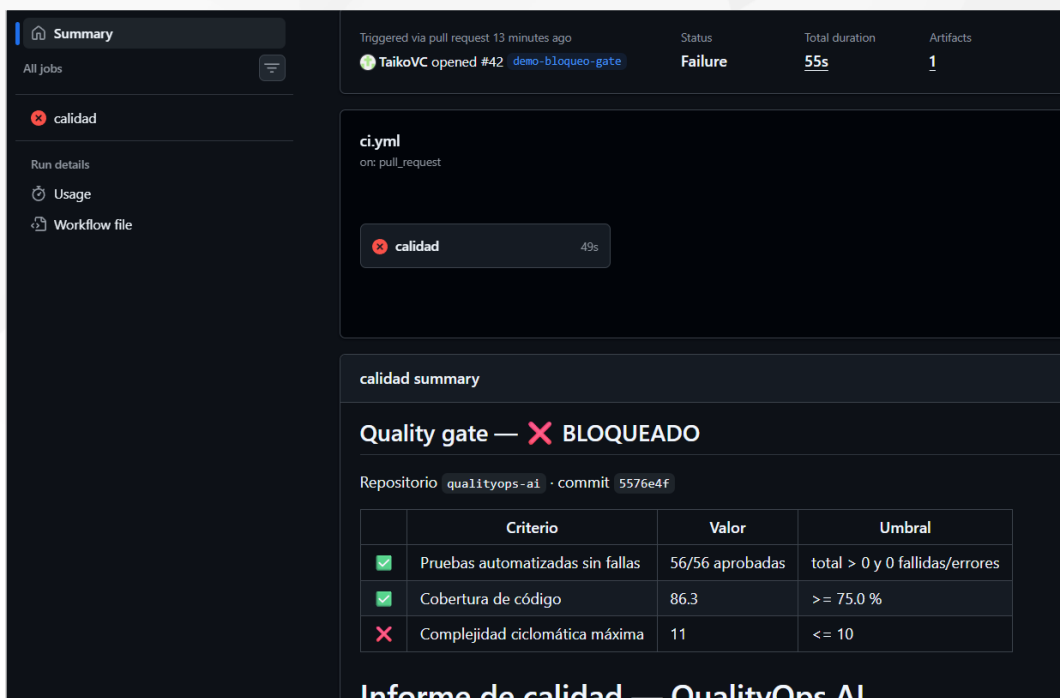


Figura 22. PR de demostración: quality gate BLOQUEADO por complejidad 11 (salida del programa, evidencia E17b)

Qué muestra. El resumen del job en rojo: pruebas y cobertura aprueban y solo falla «Complejidad ciclomática máxima: 11 (umbral ≤ 10)».

Qué demuestra. El gate detecta el problema sin intervención humana y dice exactamente qué corregir.

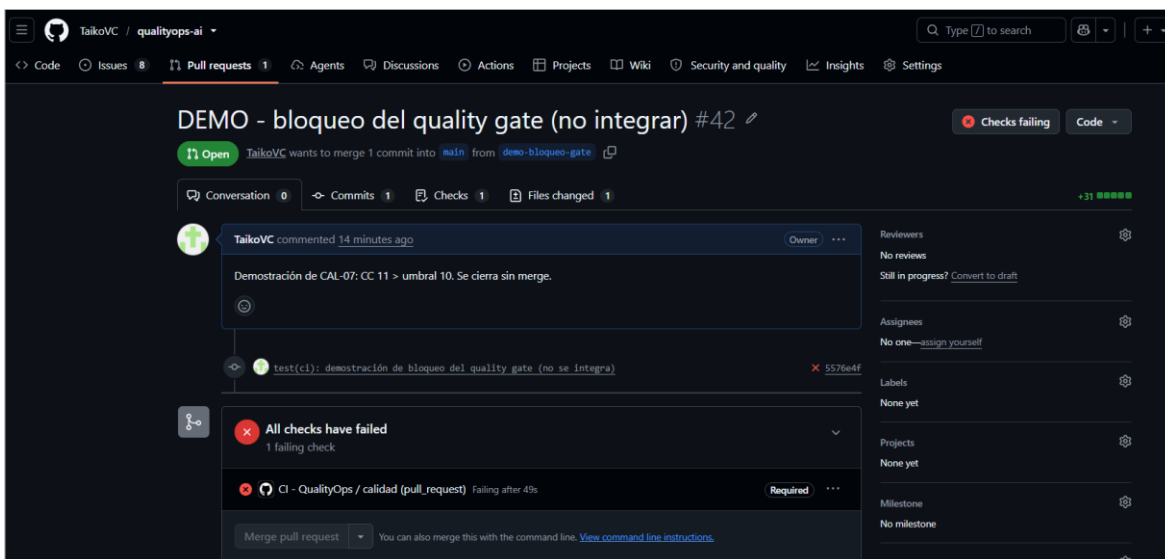


Figura 23. GitHub impide el merge del PR bloqueado (salida del programa, evidencia E17c)

Qué muestra. «All checks have failed», el check «calidad» marcado como *Required* y el botón de merge deshabilitado.

Qué demuestra. La prevención no depende de la disciplina del autor: la plataforma la impone (CAL-07).

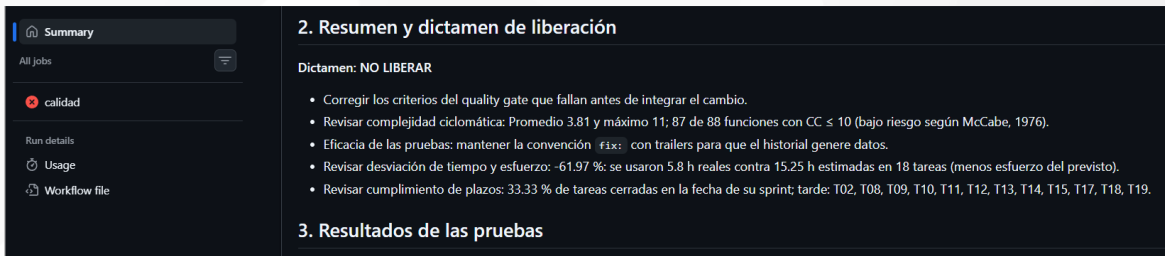


Figura 24. Informe del PR bloqueado: dictamen NO LIBERAR (salida del programa, evidencia E17d)

Qué muestra. La sección 2 del informe con el dictamen NO LIBERAR y la recomendación de corregir los criterios que fallan.

Qué demuestra. El informe se genera aunque el gate bloquee (`if: always()`) y explica la causa.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> Set-Content "$env:TEMP\umbral_demo.toml" "[tool.qualityops.umbrales]'ncobertura_minima = 95.0'"
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.quality_gate --config "$env:TEMP\umbral_demo.toml"; echo "codigo de salida: $LASTEXITCODE"

Quality gate - QualityOps AI
[OK ] Pruebas automatizadas sin fallas: 24/24 aprobadas (umbral total > 0 y 0 fallidas/errores)
[FALLA] Cobertura de código: 83.52 (umbral >= 95.0 %)
[OK ] Complejidad ciclométrica máxima: 9 (umbral <= 10)

Dictamen: BLOQUEADO

codigo de salida: 1
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.quality_gate; echo "codigo de salida: $LASTEXITCODE"

Quality gate - QualityOps AI
[OK ] Pruebas automatizadas sin fallas: 24/24 aprobadas (umbral total > 0 y 0 fallidas/errores)
[OK ] Cobertura de código: 83.52 (umbral >= 75.0 %)
[OK ] Complejidad ciclométrica máxima: 9 (umbral <= 10)

Dictamen: APROBADO

codigo de salida: 0
```

Figura 25. Bloqueo local con un umbral de demostración (captura intermedia, evidencia E08a_gate)

Captura intermedia (T09–T10). No muestra los valores finales del proyecto (están en el capítulo 3); se conserva porque prueba un comportamiento que no se repite en la ejecución final.

Qué muestra. Con un archivo de umbrales de demostración (cobertura mínima 95 %) el gate marca [FALLA], BLOQUEADO y código de salida 1; con los umbrales reales, APROBADO y código 0.

Qué demuestra. El código 1 es lo que convierte el check en rojo; la prueba se puede repetir en la demostración (capítulo 15).

10.3 Detección temprana de los defectos reales

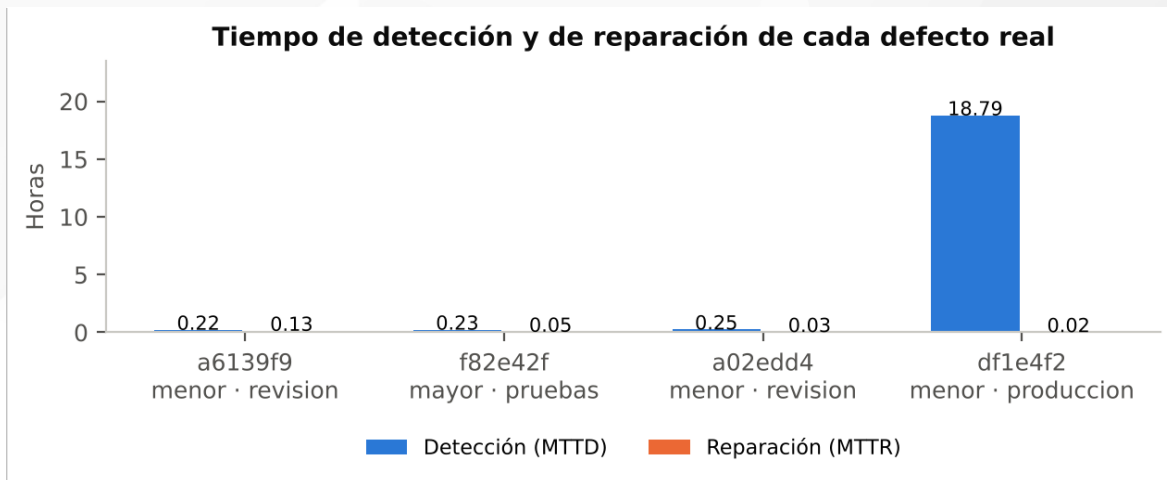


Figura 26. Tiempo de detección y de reparación de cada defecto (gráfica elaborada para el reporte)

Origen y fin. No es una pantalla del programa: la dibuja `docs/reporte/graficas.py` (función `tiempos_defectos`) con los datos de `reports/metrics.json` del commit 748d818. Su fin es comparar, defecto por defecto, cuánto vivió antes de corregirse y cuánto tardó en integrarse la corrección.

Cómo leerla. Azul = MTTD (de la introducción a la corrección); naranja = MTTR (de la corrección al merge). Debajo de cada par, el commit de corrección, la severidad y la fase.

Cómo verificarla. Los valores son los de `python -m qualityops.defects --repo .` (Figura 9).

Los defectos detectados en revisión o en pruebas vivieron minutos; el que escapó a producción (encontrado al aplicar la herramienta a otro repositorio, capítulo 14) vivió horas. Esa diferencia es justamente lo que el enfoque preventivo busca reducir.

11. CARÁCTER SISTEMÁTICO

En resumen: El proceso es sistemático porque cada cambio siguió los mismos pasos (issue → rama → commits convencionales → PR → CI → merge → registro), cada requisito se rastrea hasta su evidencia, las normas aplicadas están documentadas con su aplicación concreta y cada decisión quedó registrada con su motivo.

11.1 Kanban y ciclo de trabajo

Trabajé con Kanban en GitHub Projects (Backlog, Ready, In Progress con límite 2, Review/Validate, Done), en sprints de un día, con una issue por tarea creada desde `data/time_log.csv`. Cada tarea siguió el mismo ciclo:

1. Mover la tarjeta a *In Progress* y anotar la hora de inicio.
2. Crear la rama `TXX-descripcion` desde `main` actualizado.
3. Escribir código y pruebas; ejecutar `ruff check .` y `python -m pytest`.
4. Commits con Conventional Commits (`feat:`, `fix:` con trailers, `docs:`).
5. Abrir el PR con la plantilla y esperar el check «calidad».
6. Si hay defecto, corregirlo en la misma rama con `fix:` y `Detectado-en: revision`.
7. Merge con merge commit, borrar la rama y anotar la hora de fin.
8. Registrar evidencia, actualizar la matriz y la bitácora de IA.

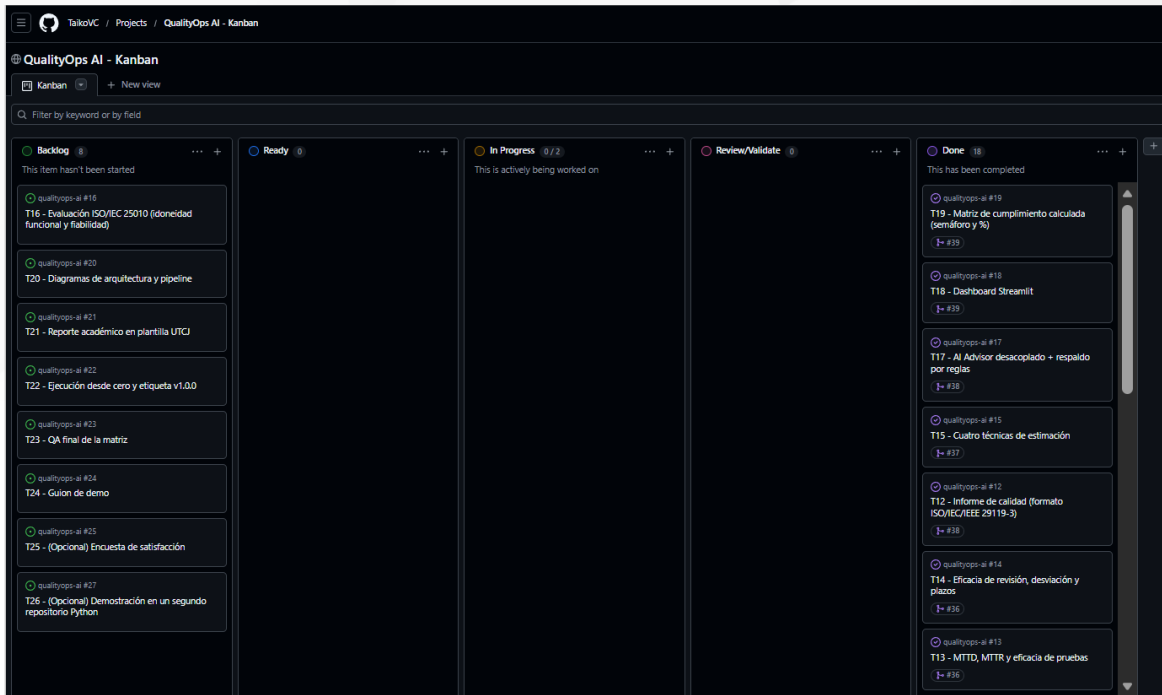


Figura 27. Tablero Kanban al cierre del desarrollo (salida del programa, evidencia E02d)

Qué muestra. 18 tareas en *Done*, cada una enlazada a su Pull Request, y en *Backlog* las de cierre y las opcionales al momento de la captura.

Qué demuestra. El flujo se sostuvo todo el proyecto y cada tarea terminada tiene un PR (PRO-01).

Tabla 34. Commits por tipo (git log, sin merges)

Tipo	Significado	Commits
feat	Funcionalidad nueva	12
docs	Documentación y registros	12
fix	Corrección de un defecto	4
chore	Mantenimiento	3
ci	Pipeline	1
Total		32

11.2 Trazabilidad de extremo a extremo

Tabla 35. Ejemplos de trazabilidad requisito → evidencia

Requisito	Tarea / issue	PR / merge	Código	Pruebas
CAL-07 Enfoque preventivo	T10 / #10	#34 / ffb14d0	quality_gate.py	test_quality_gate.py
MET-J1 Eficacia de revisión	T14 / #14	#36 / 03ca184	project_metrics.py	test_process_project_metrics.py
PRG-02 IA en frameworks	T17 / #17	#38 y #40	ai_advisor.py	test_ai_advisor_report.py
MET-P1 Complejidad	T04 / #4	#28 / 8fa7528	product_metrics.py	test_product_metrics.py

11.3 Normas aplicadas

Una norma solo se cita como aplicada si tiene una aplicación concreta y evidencia ([docs/NORMAS.md](#)).

Tabla 36. Normas aplicadas y cómo

Norma	Qué establece	Cómo se aplica
ISO/IEC 25010:2023 — Modelo de calidad del producto (SQuaRE)	9 características de calidad del producto con sus subcaracterísticas	Se evalúan idoneidad funcional (completitud, corrección, pertinencia) y fiabilidad (ausencia de fallos, disponibilidad, tolerancia a fallos, recuperabilidad), cada una con una medida real del proyecto
ISO/IEC 25023:2016 — Medición de la calidad del producto	Medidas cuantitativas para las características de 25010	Base de las fórmulas de la tabla anterior (p. ej., completitud funcional = funciones implementadas / especificadas)
IEEE 730-2014 — Procesos de aseguramiento de la calidad del software	Contenido de un plan SQA: actividades, responsables, estándares, métricas, herramientas	Plan SQA resumido: pruebas, lint, revisión por PR, quality gates, métricas y registros
ISO/IEC/IEEE 29119-3:2021 — Documentación de pruebas	Estructura de documentos de prueba, incluido el informe de finalización de pruebas	Formato fijo del informe de calidad que genera el programa en cada ejecución
ISO/IEC/IEEE 12207:2017 — Procesos del ciclo de vida del software	Procesos técnicos y de gestión del ciclo de vida	Mapeo de cada etapa del pipeline (desarrollo, verificación, integración, gestión de configuración, liberación) a sus procesos
ISO/IEC/IEEE 90003:2018 — Aplicación de ISO 9001:2015 al software	Gestión de calidad basada en procesos, prevención y mejora continua	Fundamento del enfoque preventivo (quality gates antes de integrar) y de la mejora continua (métricas → acciones)
ISO/IEC 20926:2009 — Método IFPUG de medición del tamaño funcional	Conteo de puntos de función (EI, EO, EQ, ILF, EIF)	Técnica de estimación por puntos de función

Tabla 37. Procesos de ISO/IEC/IEEE 12207 y su práctica en el proyecto

Proceso	Práctica
Planificación y control del proyecto	Backlog con estimación previa; desviación y plazos medidos en cada ejecución
Gestión de decisiones	docs/DECISIONES.md (D01–D21) con motivo y alternativa
Gestión de la configuración	Git, una rama por tarea, merge commits, ruleset, versiones fijas y tags
Medición	Motor de métricas de producto, proceso y proyecto
Aseguramiento de la calidad	Plan SQA (cap. 8) y matriz de cumplimiento
Verificación y validación	ruff, pytest, comparación con radon/coverage, gate y dictamen
Transición / liberación	Etiquetas v1.0.0 y v1.0.1

11.4 Uso de IA durante el desarrollo

Usé Claude (Anthropic) como copiloto técnico bajo la regla «la IA propone; el autor ejecuta, verifica la salida real y decide». Cada uso quedó en [docs/AI_LOG.md](#), incluidos los errores de la propia IA: propuso un proveedor de modelos (GitHub Models) retirado meses antes, lo que el CI reveló como defecto real y se corrigió cambiando a Gemini (decisiones D17–D19).

12. MÉTRICAS CALCULADAS APLICANDO IA

En resumen: Las nueve métricas se calculan con código determinista y la IA las verifica e interpreta. Producto: complejidad promedio 3.77 (máx. 9), cobertura 88.79 %, densidad 3.73 def/KLOC. Proceso: MTTD 0.24 h, MTTR 0.04 h, eficacia de pruebas 75.0 % (sin críticos). Proyecto: eficacia de revisión 50.0 %, desviación -63.66 %, plazos 27.27 %.

12.1 Cómo participa la IA

Tabla 38. Reparto de responsabilidades entre el código y la IA

Tarea	Quién la hace	Por qué
Calcular cada métrica	Código determinista (radon, pytest-cov, git, fórmulas)	Reproducible: mismo commit, mismo número
Comprobar la consistencia	AI Advisor: 5 verificaciones de coherencia	Detecta errores antes de publicarlos
Valorar cada métrica	AI Advisor: reglas con referencias	Siempre disponible, sin costo
Redactar interpretación, riesgos y acciones	LLM (gemini) con prompt fijo	Comunica a personas no técnicas

El prompt indica al modelo: «Con las métricas siguientes (ya calculadas; NO las recalculas ni inventes otras) escribe en español: 1) una interpretación breve de cada métrica citando su referencia, 2) los 3 riesgos principales y 3) 3 acciones concretas. Si un valor es null o n = 0, dilo explícitamente.»

La IA también se equivoca, por eso no calcula. En una ejecución intermedia (commit 71f84cb, registrada en [docs/AI_LOG.md](#)) el modelo escribió que hubo «3 defectos críticos con fase conocida» cuando eran 0; en otras ejecuciones lo reportó bien, lo que muestra que su respuesta no es determinista. Las cifras del informe nunca cambian porque vienen del código (decisión D02).

12.2 Métricas de producto

Complejidad ciclomática = decisiones + 1, por función (McCabe, 1976: ≤ 10 bajo riesgo). Resultado: 87 funciones, promedio 3.77, mediana 3, máximo 9; 87 de 87 en bajo riesgo.

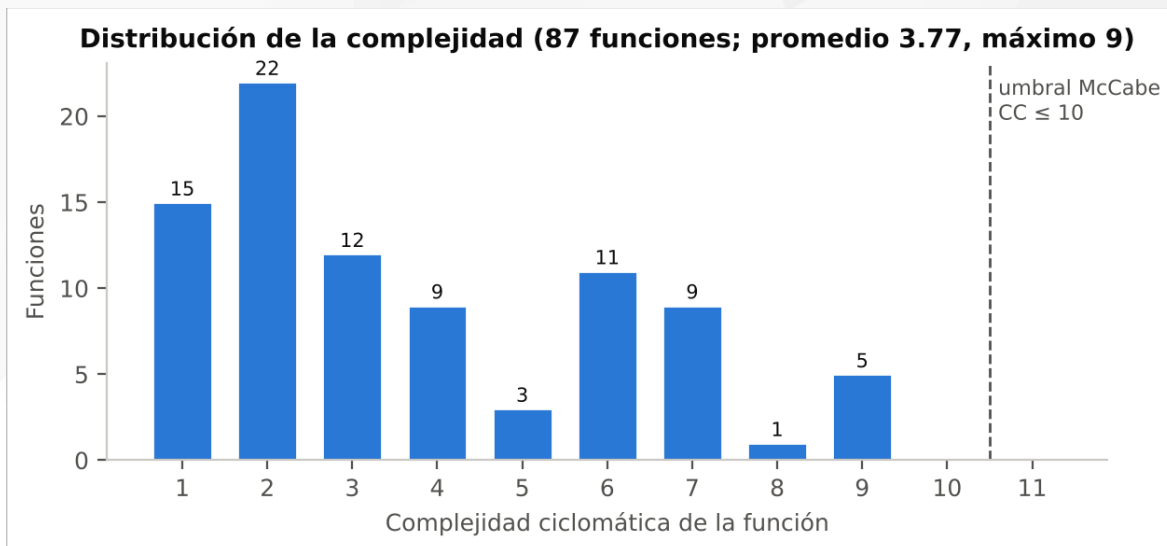


Figura 28. Distribución de la complejidad por función (gráfica elaborada para el reporte)

Origen y fin. No es una pantalla del programa: la dibuja `docs/reporte/graficas.py` (función `distribucion_complejidad`) con los datos de `reports/metrics.json` del commit 748d818. Su fin es mostrar si la complejidad está concentrada en pocas funciones o repartida.

Cómo leerla. Cuántas funciones tienen cada valor de complejidad; la línea punteada es el umbral. La mayoría tiene 1 o 2 y ninguna lo cruza.

Cómo verificarla. Las funciones y sus valores son los de `python -m qualityops.product_metrics --repo .` y la tabla «Funciones más complejas» del dashboard (Figura 7).

Cobertura de código = líneas ejecutadas / ejecutables × 100 (Google Testing Blog: 75 % encomiable). Resultado: 697 de 785 = **88.79 %**; el desglose por archivo está en el capítulo 8.

Densidad de defectos = defectos de producto / KLOC (McConnell, 2004: 1 a 25 en software entregado). Resultado: 4 / 1.073 = **3.73 def/KLOC**.

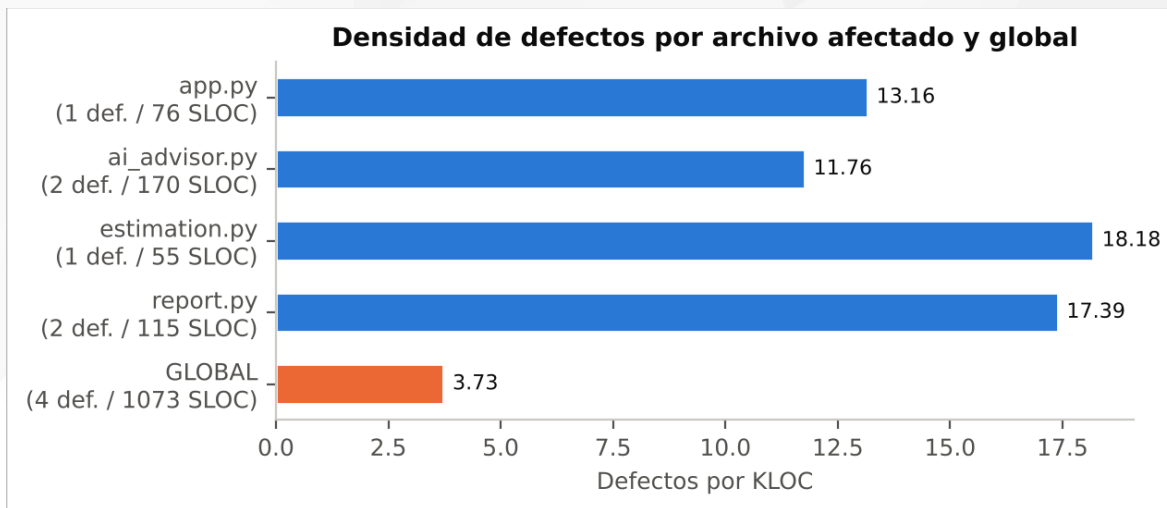


Figura 29. Densidad de defectos por archivo afectado y global (gráfica elaborada para el reporte)

Origen y fin. No es una pantalla del programa: la dibuja `docs/reporte/graficas.py` (función `densidad_por_archivo`) con los datos de `reports/metrics.json` del commit 748d818. Su fin es señalar dónde se concentran los defectos.

Cómo leerla. Un defecto en un archivo pequeño pesa mucho. Los defectos se concentran en la capa de IA y presentación, la de mayor cambio de alcance; el motor de cálculo casi no tuvo.

Cómo verificarla. Los valores son los de `python -m qualityops.density --repo ..`

12.3 Métricas de proceso

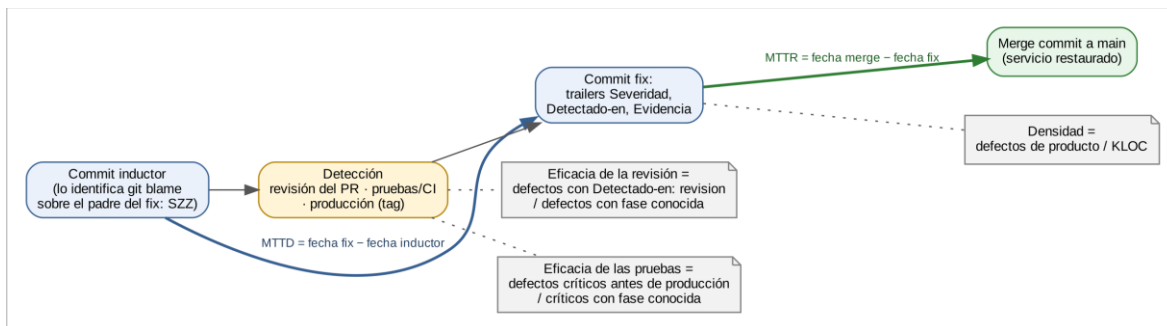


Figura 30. Ciclo de vida de un defecto y dónde se mide cada métrica (diagrama de elaboración propia)

Origen y fin. Diagrama dibujado con Graphviz desde `docs/diagramas/ciclo_defecto.dot`; no es una salida del programa. Su fin es explicar de qué fechas del historial sale cada métrica de proceso.

Cómo leerlo. Un defecto nace en un commit inductor, se detecta en una fase, se corrige con `fix:` y se integra con un merge. MTTD va de la introducción a la corrección y MTTR de la corrección al merge.

Cómo verificarlo. Las fechas de cada defecto aparecen en la salida de `qualityops.defects` (capítulo 3).

Tabla 39. Métricas de proceso de la ejecución final

Métrica	Fórmula	Resultado
MTTD	promedio y mediana de (fix – inductor)	Promedio 4.87 h, mediana 0.24 h (n = 4)
MTTR	promedio y mediana de (merge – fix)	Promedio 0.06 h, mediana 0.04 h (n = 4)
Eficacia de las pruebas (críticos)	críticos antes de producción / críticos con fase conocida	Sin datos (0/0)
Eficacia de las pruebas (todas)	antes de producción / con fase conocida	75.0 % (3/4)

La eficacia sobre errores **críticos** es la que pide la actividad y el programa la calcula, pero no hubo defectos críticos: reportar 100 % sería engañoso y 0 % falso, por eso es «sin datos» y se muestra también la de todas las severidades.

12.4 Métricas de proyecto

Eficacia de la revisión = defectos detectados en revisión / con fase conocida: 2 de 4 = **50.0 %**. **Desviación de tiempo y esfuerzo** = (real – estimado) / estimado, por módulo: 22 tareas, 20.75 h estimadas contra 7.54 h reales = **-63.66 %**. **Cumplimiento de plazos** = tareas cerradas en la fecha de su sprint: 6 de 22 = **27.27 %**.

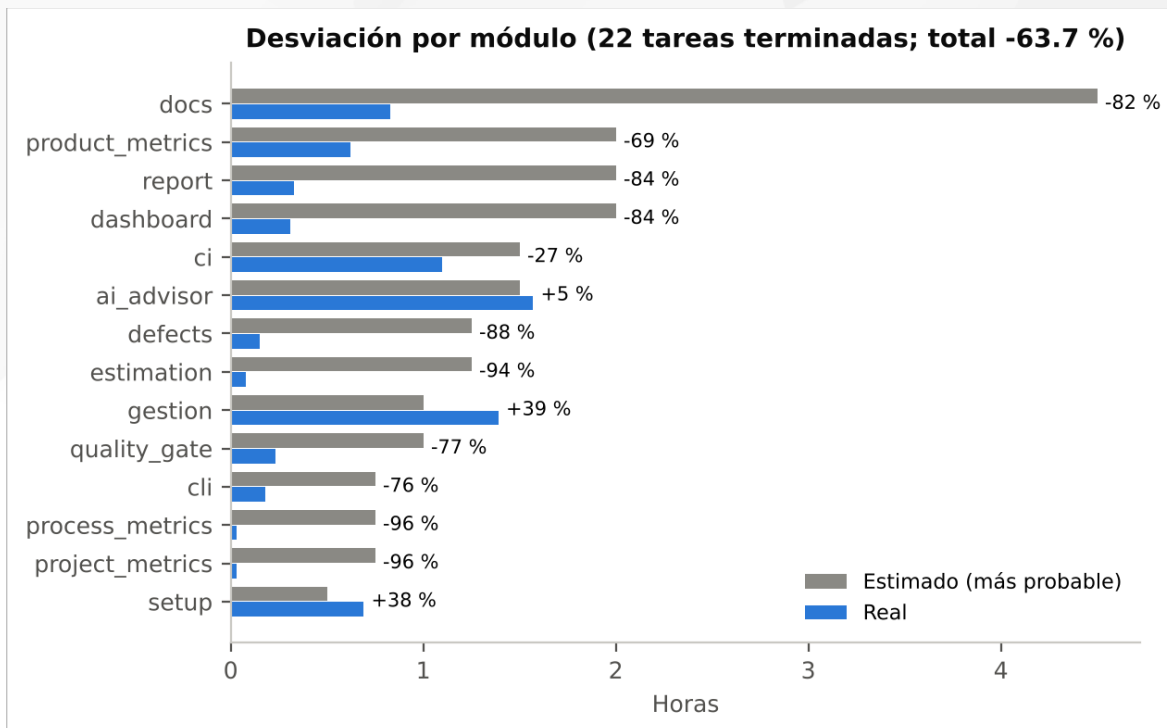


Figura 31. Horas estimadas y reales por módulo (gráfica elaborada para el reporte)

Origen y fin. No es una pantalla del programa: la dibuja `docs/reporte/graficas.py` (función `desviacion_por_modulo`) con los datos de `reports/metrics.json` del commit 748d818. Su fin es comparar módulo por módulo el esfuerzo planeado con el real y relacionarlo con los defectos.

Cómo leerla. Tardaron más de lo estimado: `setup` (+38 %), `gestion` (+39 %), `ai_advisor` (+5 %). `setup` y `gestion` fueron tareas manuales (crear el repositorio, moverlo fuera de System32, configurar el tablero); `ai_advisor` incluye el retrabajo de T17b (cambio de proveedor de IA). Los demás módulos tardaron entre 27 % y 96 % menos, porque el código lo generó la IA y el tiempo registrado es el de integrar y verificar (ver amenazas a la validez). Los módulos con más ahorro no tuvieron defectos; los defectos aparecieron en la capa de IA y presentación, la de mayor cambio de alcance.

Cómo verificarla. Los valores por módulo son los de la pestaña «Proyecto» del dashboard (Figura 11).

Las fechas planeadas asumían una dedicación diaria que no se sostuvo el 6 y el 7 de octubre; el trabajo se recuperó el 8. Por eso el esfuerzo fue menor al estimado y aun así los plazos se incumplieron: son dos métricas distintas.

13. TÉCNICAS DE ESTIMACIÓN

En resumen: Estimé el proyecto antes de escribir código (commit 76f14fb). Juicio de expertos: 22.5 h. Tres puntos: 24.79 ± 1.35 h. Puntos de función: $97 \text{ PF} \times 0.0437 \text{ h/PF} = 4.24 \text{ h}$. Análoga: 4.79 h. Las dos últimas, basadas en mi productividad real con IA, fueron las más cercanas a las horas registradas.

13.1 Juicio de expertos y tres puntos

Para cada tarea estimé tres valores: optimista (O), más probable (M) y pesimista (P). El **juicio de expertos** es $\Sigma M = 22.5 \text{ h}$. La **estimación de tres puntos** (PERT) usa $E = (O + 4M + P)/6$ y $\sigma = (P - O)/6$ por tarea (PMI, 2017): $E = 24.79 \text{ h}$, $\sigma = 1.35 \text{ h}$, con 95 % de confianza entre 22.09 y 27.5 h.

Tabla 40. Línea base de estimación por tarea (horas)

Tarea	Descripción	O	M	P	PERT	Real
T01	Repositorio, estructura, entorno y README	0.25	0.5	1.0	0.54	0.69
T02	Tablero Kanban (GitHub Projects) e issues	0.25	0.5	1.0	0.54	0.70
T03	Estimación inicial y plantillas de datos	0.25	0.5	1.0	0.54	0.69
T04	Complejidad ciclomática y LOC (radon)	0.5	1.0	2.0	1.08	0.27
T05	Cobertura de código (pytest-cov)	0.5	0.75	1.5	0.83	0.25
T06	Minería de defectos SZZ adaptada (git + trailers)	0.75	1.25	2.5	1.38	0.15
T07	Densidad de defectos	0.25	0.25	0.5	0.29	0.10
T08	CLI python -m qualityops y metrics.json	0.5	0.75	1.5	0.83	0.18
T09	Configuración ruff/pytest/coverage (pyproject)	0.25	0.25	0.5	0.29	0.06
T10	Quality gate con umbrales	0.5	0.75	1.5	0.83	0.17
T11	Pipeline GitHub Actions	0.5	1.0	2.0	1.08	0.28
T12	Informe de calidad (formato ISO/IEC/IEEE 29119-3)	0.5	0.75	1.5	0.83	0.12
T13	MTTD, MTTR y eficacia de pruebas	0.5	0.75	1.5	0.83	0.03
T14	Eficacia de revisión, desviación y plazos	0.5	0.75	1.5	0.83	0.03
T15	Cuatro técnicas de estimación	0.75	1.25	2.5	1.38	0.08
T16	Evaluación ISO/IEC 25010 (idoneidad funcional y fiabilidad)	0.25	0.5	1.0	0.54	0.09
T17	AI Advisor desacoplado + respaldo por reglas	1.0	1.5	3.0	1.67	1.57
T18	Dashboard Streamlit	1.25	2.0	4.0	2.21	0.31
T19	Matriz de cumplimiento calculada (semáforo y %)	0.5	0.75	1.5	0.83	0.12
T20	Diagramas de arquitectura y pipeline	0.25	0.5	1.0	0.54	0.09
T21	Reporte académico en plantilla UTCJ	2.5	4.0	8.0	4.42	0.74
T22	Ejecución desde cero y etiqueta v1.0.0	0.25	0.5	1.0	0.54	—
T23	QA final de la matriz	0.5	1.0	2.0	1.08	—
T24	Guion de demo	0.25	0.25	0.5	0.29	—
T25	(Opcional) Encuesta de satisfacción	0.25	0.5	1.0	0.54	—

13.2 Puntos de función y estimación análoga

Tabla 41. Conteo de puntos de función IFPUG (data/estimacion.json)

Tipo	Complejidad	Cantidad	Peso	PF	Funciones
EI	baja	2	3	6	Ejecutar análisis (python -m qualityops --repo --salida); ejecutar quality gate (--metricas --config)
EO	media	4	5	20	metrics.json con métricas derivadas; dictamen del quality gate + resumen en Actions; informe de calidad; interpretación del AI Advisor
EO	media	5	5	25	Vistas del dashboard: resumen/cumplimiento, producto, proceso, proyecto, estimación

Tipo	Complejidad	Cantidad	Peso	PF	Funciones
EQ	baja	4	3	12	Consultas por módulo: complejidad/LOC, cobertura, defectos, densidad
ILF	baja	1	7	7	reports/ (metrics.json + gate.json), mantenido por la herramienta
EIF	media	1	7	7	Historial de Git (commits, merges, tags, blame)
EIF	baja	4	5	20	Reportes de pytest/coverage; data/time_log.csv; pyproject.toml (umbrales y calendario); data/estimacion.json
Total				97	Puntos de función sin ajustar

Puntos de función (ISO/IEC 20926): la tasa no se tomó de la industria (no hay una publicada para desarrollo con IA) sino de mi proyecto de referencia: 4.5 h / 103 PF = 0.0437 h/PF, así que $97 \times 0.0437 = 4.24$ h. **Análoga**: la referencia es la herramienta [metrics](#) que desarrollé con IA en septiembre (4.5 h, 103 PF); escalada por tamaño y con un factor de 1.13 (entorno más simple $\times 0.75$, proceso más pesado $\times 1.5$) da $4.5 \times (97/103) \times 1.13 = 4.79$ h.

13.3 Comparación con las horas reales

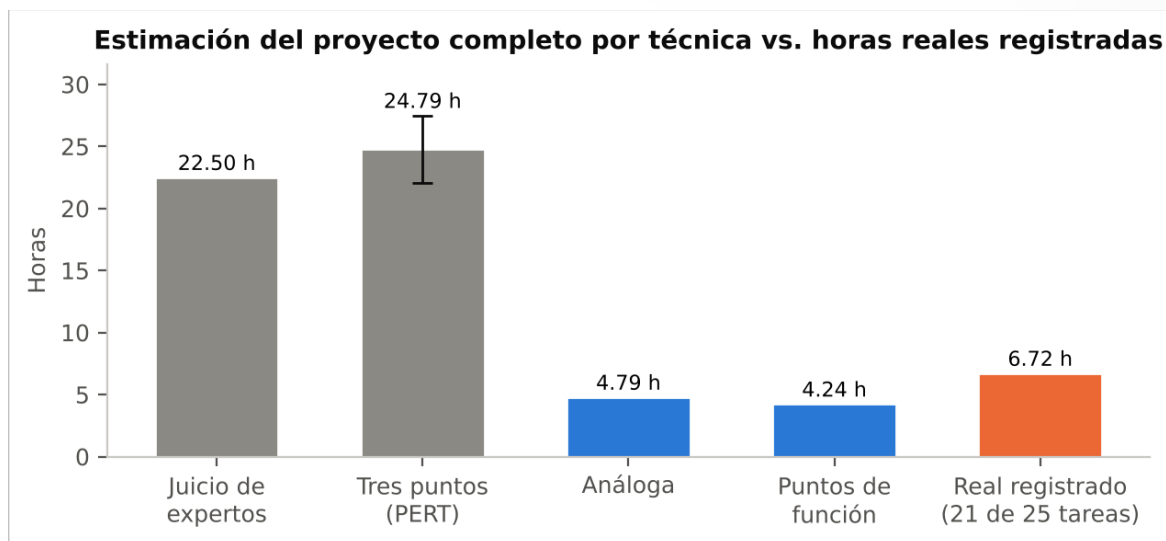


Figura 32. Estimación por técnica frente a horas reales (gráfica elaborada para el reporte)

Origen y fin. No es una pantalla del programa: la dibuja [docs/reporte/graficas.py](#) (función [estimaciones_vs_real](#)) con los datos de [reports/metrics.json](#) del commit 748d818. Su fin es comparar las cuatro técnicas con lo que realmente tomó el trabajo registrado.

Cómo leerla. Juicio y PERT (gris) suponían programar a mano; análoga y PF (azul) usan la productividad real con IA y quedaron cerca de lo registrado (naranja). La barra de PERT incluye su rango del 95 %.

Cómo verificarla. Las cuatro estimaciones son las de la línea «Estimación» de la CLI (Figura 3); las horas reales son la suma de la columna `real_h` de `data/time_log.csv`.

14. IMPLEMENTACIÓN EN OTRO REPOSITORIO

En resumen: QualityOps AI funciona con cualquier repositorio Python con Git y pruebas pytest, sin modificarlo. Se usa de dos formas: **local** (cinco comandos) o **en el CI** del repositorio (copiar un archivo YAML). Cada persona usa su **propia** llave de IA, que además es opcional. La guía completa está en `docs/GUIA_DE_USO.md` y la probé sobre un repositorio público ajeno, donde encontré el defecto 5.

14.1 Requisitos y consideraciones

Tabla 42. Qué necesita el repositorio que se va a analizar

Requisito	Por qué	Si no se cumple
Código Python	Complejidad y líneas se miden con radon	No hay métricas de producto
Pruebas que corran con <code>python -m pytest</code> desde la raíz	La cobertura se mide ejecutándolas	Pruebas = 0 y el gate bloquea
Dependencias del proyecto en el mismo Python	Las pruebas usan ese intérprete	Pruebas con <code>ModuleNotFoundError</code>
Historial Git completo	La minería lee commits, merges y tags	Defectos y tiempos «sin datos»
Correcciones con <code>fix: ...</code> (y trailers opcionales)	Así se reconocen los defectos	Defectos = 0 (no se inventan)
Herramienta en una carpeta aparte	Para no medir el código de QualityOps	Las métricas mezclan ambos proyectos

14.2 Uso local

Análisis local de cualquier repositorio (PowerShell)

```
# 1) Descargar la herramienta en una carpeta APARTE de tu proyecto
git clone https://github.com/TaikoVC/qualityops-ai.git
cd qualityops-ai
python -m venv .venv
.\venv\Scripts\Activate.ps1           # Linux/macOS: source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pip install -r C:\ruta\proyecto\requirements.txt # dependencias de TUS pruebas

# 2) Medir, validar e informar
python -m qualityops --repo C:\ruta\proyecto --salida ..\reports_proyecto
python -m qualityops.quality_gate --metricas ..\reports_proyecto\metrics.json --config
C:\ruta\proyecto\pyproject.toml
python -m qualityops.report --metricas ..\reports_proyecto\metrics.json

# 3) Ver los resultados en el dashboard
$env:QUALITYOPS_REPORTS = "..\reports_proyecto"
streamlit run app.py
```

Tabla 43. Dónde ver cada resultado

Resultado	Dónde
Resumen de todas las métricas	Terminal y <code>metrics.json</code>
Dictamen del gate	Terminal, <code>gate.json</code> y código de salida (0 aprobado, 1 bloqueado)
Informe de 12 secciones y dictamen de liberación	<code>quality_report.md</code>
Interpretación y verificaciones de coherencia	<code>ai_analysis.json</code> y sección 10 del informe
Vista gráfica	Dashboard en <code>http://localhost:8501</code>
En el CI	Pestaña <i>Actions</i> → run → <i>Summary</i> y artefacto <code>reportes-calidad</code>

14.3 Integración al CI del repositorio

1. Copiar `docs/ejemplos/qualityops-ci.yml` al repositorio como `.github/workflows/qualityops.yml`.
2. Revisar las dependencias: el workflow instala `requirements.txt` o, si no existe, el propio paquete con `pip install -e`.
3. Revisar `on:`: corre en cada Pull Request y en cada push a `main` o `master` (ajustar si la rama principal se llama distinto).
4. Abrir un Pull Request: el job «calidad» corre solo y publica el gate y el informe en el *Summary*.
5. Recomendado: en *Settings* → *Rules* → *Rulesets*, exigir PR y el check «calidad» para integrar a `main`.
6. Opcional: guardar una llave propia de Gemini como secreto con `gh secret set GEMINI_API_KEY`.

La plantilla descarga el proyecto y la herramienta en **carpetas separadas** (`proyecto/` y `qualityops-ai/`) para no mezclar su código, y fija la herramienta en una **versión etiquetada** (ref: `v1.0.1`) para que los resultados no cambien si QualityOps se actualiza (decisión D21). Los umbrales se leen del `pyproject.toml` del proyecto (`[tool.qualityops.umbrales]`); si no existen se usan 75 % y CC 10.

14.4 La llave de IA

- **Cada usuario usa su propia llave.** No se comparte ni se reutiliza la de otra persona: identifica a su dueño y consume su cuota. Los secretos de un repositorio no se copian a sus clones ni a sus *forks*.

- **Es gratuita y opcional:** se crea en Google AI Studio (*Get API key*). Sin llave, el informe se genera por reglas y lo indica en la fila «Intento de LLM»; las métricas nunca dependen de la IA.
- **Local:** `$env:GEMINI_API_KEY = Read-Host "Pega tu llave"` (no queda en ningún archivo). **En CI:** `gh secret set GEMINI_API_KEY`; el workflow la lee de `secrets.GEMINI_API_KEY`.
- Solo se envían métricas agregadas, nunca código ni datos personales.

14.5 Demostración en un repositorio ajeno

Apliqué la herramienta a **python-slugify** (github.com/un33k/python-slugify), una librería pública con pruebas y más de 300 commits, siguiendo exactamente el uso local. Además de confirmar que la herramienta funciona fuera de su propio repositorio, la prueba reveló un defecto real.

```

Warning: In the working copy of tests/test_estimation.py, L1 will be replaced by CML the next time git touches it
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops --repo ../python-slugify --salida ../reports_slugify
QualityOps AI 0.1.0 - python-slugify @ f0010f9
Pruebas:      269/274 aprobadas · 0 fallidas · 0 errores
Cobertura:    98.12 %
Complejidad:  promedio 10.95 · máximo 34
Tamaño:       0.732 KLOC en 10 archivos
Defectos:    0 · densidad 0.0 def/KLOC
MTTD / MTTR:  None h / None h (n = 0 / 0)
Desviación:   None % (0 tareas terminadas)
Plazos:       None % a tiempo
Estimación:   juicio 0 h · PERT 0.0 h · PF None h · análoga None h

Resultados guardados en ../reports_slugify/metrics.json
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai>

```

Figura 33. Defecto 5: estimación «0 h» en un repositorio sin registro de tiempos (v1.0.0) (salida del programa, evidencia E18a)

Qué muestra. Con el código de la versión v1.0.0, el resumen de python-slugify mostraba «juicio 0 h · PERT 0.0 h» aunque el proyecto no tiene `data/time_log.csv`.

Qué demuestra. El programa inventaba un valor donde no había datos. Se corrigió (ahora reporta `None/«—»`), con pruebas nuevas, y se publicó como v1.0.1; por haber escapado a una versión liberada cuenta como defecto en producción.

```
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m pytest -q
..... [ 96%]
..... [100%]
58 passed in 34.47s
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> ruff check .
All checks passed!
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops --repo ..\python-slugify --salida ..\reports_slugify
QualityOps AI 0.1.0 - python-slugify @ f0010f9
Pruebas: 269/274 aprobadas · 0 fallidas · 0 errores
Cobertura: 98.12 %
Complejidad: promedio 10.95 · máximo 34
Tamaño: 0.732 KLOC en 10 archivos
Defectos: 0 · densidad 0.0 def/KLOC
MTTD / MTTR: None h / None h (n = 0 / 0)
Desviación: None % (0 tareas terminadas)
Plazos: None % a tiempo
Estimación: juicio None h · PERT None h · PF None h · análoga None h

Resultados guardados en ..\reports_slugify\metrics.json
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.quality_gate --metricas ..\reports_slugify\metrics.json --confi
g ..\python-slugify\pyproject.toml

Quality gate - QualityOps AI
[OK ] Pruebas automatizadas sin fallas: 269/274 aprobadas (umbral total > 0 y 0 fallidas/errores)
[OK ] Cobertura de código: 98.12 (umbral >= 75.0 %)
[FALLA] Complejidad ciclomática máxima: 34 (umbral <= 10)

Dictamen: BLOQUEADO

(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> python -m qualityops.report --metricas ..\reports_slugify\metrics.json
Informe generado: ..\reports_slugify\quality_report.md · dictamen: NO LIBERAR · interpretación: reglas
(.venv) PS C:\Users\LPC\proyectos\qualityops-ai> |
```

Figura 34. python-slugify: quality gate BLOQUEADO por complejidad (salida del programa, evidencia E18b)

Qué muestra. Con la corrección: pruebas y cobertura del proyecto ajeno aprueban, pero su función más compleja tiene complejidad 34, así que el gate bloquea y el informe dictamina NO LIBERAR.

Qué demuestra. La herramienta valida código que no es suyo, con sus mismos criterios (RNF-06).



Figura 35. python-slugify en el dashboard de QualityOps (salida del programa, evidencia E18c)

Qué muestra. La pestaña «Producto» del dashboard apuntando a los reportes de python-slugify (QUALITYOPS_REPORTS).

Qué demuestra. Todos los resultados de otro repositorio se exploran igual que los del propio proyecto.

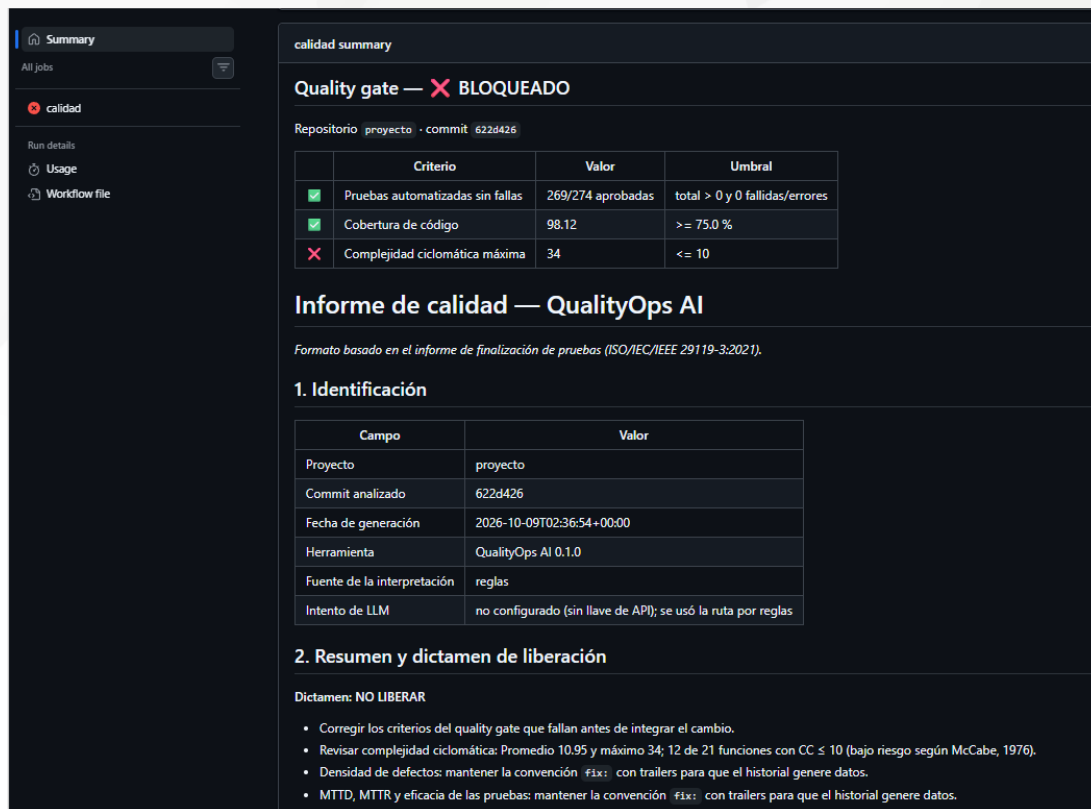


Figura 36. Plantilla de workflow en el CI de otro repositorio (fork de python-slugify) (salida del programa, evidencia E19)

Qué muestra. El Summary del job «calidad» en TaikoVC/python-slugify tras agregar `qualityops-ci.yml` en un PR: 269/274 pruebas y 98.12 % de cobertura aprueban, la complejidad máxima de 34 falla, el gate dice BLOQUEADO y el informe NO LIBERAR. Como el fork no tiene llave, la interpretación es por reglas («no configurado»).

Qué demuestra. La integración al CI de un repositorio ajeno funciona copiando un solo archivo, sin llave de IA y con los mismos criterios que en el proyecto propio (PRG-08, RNF-06).

14.6 Configurar los umbrales

Cada proyecto define sus umbrales en su propio `pyproject.toml`, sin tocar el código de la herramienta. Por ejemplo, docker/secrets-engine permite complejidad 16 en su linter; un proyecto así puede empezar con:

```
pyproject.toml del proyecto analizado
[tool.qualityops.umbrales]
cobertura_minima = 75.0
complejidad_maxima = 16 # justificado en DECISIONES; bajar a 10 al refactorizar
```

Recomiendo mantener 10 (el valor de McCabe) en proyectos nuevos, como QualityOps, y subirlo solo en proyectos existentes con una justificación escrita, para que el gate no bloquee todo el primer día pero impida que la complejidad siga creciendo.

15. REPRODUCCIÓN DE RESULTADOS Y SALIDAS POSIBLES

En resumen: Guía para poner a prueba el programa frente al profesor: qué comando ejecutar, qué salida esperar en cada escenario y en qué figura del reporte se ve esa misma salida.

Tabla 44. Escenarios de prueba y salida esperada

Escenario	Cómo provocarlo	Salida esperada	Dónde se ve
Gate aprobado	<code>python -m qualityops.quality_gate</code>	3 criterios [OK], APROBADO, código 0	Figura 3
Gate bloqueado (local)	<code>--config</code> con cobertura mínima 95	[FALLA], BLOQUEADO, código 1	Figura 25
Gate bloqueado (CI)	PR con una función de CC 11	Check rojo, merge deshabilitado, NO LIBERAR	Figura 22
Informe con LLM	Llave cargada + <code>qualityops.report</code>	interpretación: <code>llm</code> , «Intento de LLM: usado»	Figura 4
Informe sin llave	Sin <code>GEMINI_API_KEY</code>	interpretación: reglas, «no configurado»	Sección 1 del informe
Falla del LLM	Modelo inexistente o servicio caído	fuelle: reglas y <code>error_llm</code> con el motivo	Figura 21
Liberar con condiciones	Gate aprobado y una métrica sin datos	LIBERAR CON CONDICIONES	Figura 4
Defectos	<code>python -m qualityops.defects --repo .</code>	Defectos con severidad, fase, MTDD, MTTR	Figura 9
Otro repositorio	<code>python -m qualityops --repo <ruta></code>	Métricas del otro proyecto	Figura 34

Comandos para reproducir el bloqueo y las dos rutas de interpretación (PowerShell)

```
# Gate con umbral de demostración (debe BLOQUEAR) y con el real (debe APROBAR)
Set-Content "$env:TEMP\umbral_demo.toml" "[tool.qualityops.umbrales]`ncobertura_minima = 95.0"
python -m qualityops.quality_gate --config "$env:TEMP\umbral_demo.toml"; echo "codigo: $LASTEXITCODE"
python -m qualityops.quality_gate; echo "codigo: $LASTEXITCODE"

# Informe sin IA (ruta por reglas) y con IA
Remove-Item Env:GEMINI_API_KEY -ErrorAction SilentlyContinue; python -m qualityops.report
$env:GEMINI_API_KEY = Read-Host "Pega tu llave"; python -m qualityops.report
```

Orden recomendado para reproducir todo: 1) `python -m pytest -q` y `ruff check .`; 2) métricas, gate e informe (capítulo 3); 3) `python -m qualityops.defects --repo .` y `git log -1 <hash>` para ver los trailers de un defecto; 4) gate con el umbral de demostración; 5) dashboard; 6) pestaña *Actions* del repositorio para el pipeline y el PR #42 bloqueado.

16. AMENAZAS A LA VALIDEZ

En resumen: Todas las métricas se calcularon con datos reales y son reproducibles; sus límites principales son una muestra pequeña de defectos sin ninguno crítico y un tiempo real que no incluye el tiempo en que la IA generó el código.

- **Muestra pequeña.** Con $n = 4$ defectos, MTDD y MTTR son indicativos. No hay defectos críticos, por lo que la eficacia sobre críticos no se puede calcular.
- **Tiempo real sin la generación de código.** El reloj de cada tarea empezaba con el código generado por la IA; mide integración, verificación y PR. Eso explica la desviación negativa.
- **SZZ es una aproximación.** Un defecto se atribuye al commit más reciente que tocó las líneas corregidas, que no siempre es el que lo introdujo.
- **MTDD hasta la corrección.** No hay marca separada de detección; se usa la fecha del commit `fix:`.
- **Bloques conjuntos.** Algunas tareas se hicieron juntas y su tiempo se repartió según su estimación (D11, D15).
- **Un solo autor y un proyecto pequeño.** La demostración con python-slugify muestra que la herramienta funciona en otros repositorios, pero las cifras de este reporte describen solo este proyecto.

III. CONCLUSIÓN

Desarrollé QualityOps AI como una pieza de infraestructura DevOps, no como una aplicación de negocio: su trabajo es decidir, en cada Pull Request, si un cambio tiene la calidad suficiente para integrarse. La ejecución final lo confirma con datos reales: 58 pruebas en verde, cobertura de 88.79 %, ninguna función por encima de complejidad 10, 4 defectos de producto registrados y corregidos con un tiempo mediano de restauración de 0.04 h, y un quality gate que la plataforma hace obligatorio.

Lo más valioso fue la forma de obtener cada número: al sacar los defectos del historial de Git y los tiempos de un registro hecho antes del código, cualquier cifra se reproduce con un comando. Esa disciplina expuso los errores propios (cinco defectos reales, uno de ellos encontrado al usar la herramienta en otro repositorio) y los de la IA (un proveedor retirado y una interpretación con un dato confundido); en todos los casos la verificación automática impidió que el error llegara al resultado.

La IA aceleró el desarrollo y hace comprensibles las métricas, pero necesitó supervisión; por eso el diseño separa el cálculo determinista de la interpretación generativa. Como siguientes pasos propongo registrar la hora de detección de cada defecto para medir el MTDD con más precisión y publicar la herramienta como una *GitHub Action* reutilizable para que otros equipos la integren con una sola línea. También se podría activar Dependabot para proponer actualizaciones de las dependencias fijadas; no lo incluí porque las versiones fijas son intencionales (reproducibilidad) y cada actualización pasaría de todos modos por el quality gate.

IV. REFERENCIAS

- Arguelles, C., Ivanković, M., & Bender, A. (2020, 7 de agosto). *Code coverage best practices*. Google Testing Blog. <https://testing.googleblog.com/2020/08/code-coverage-best-practices.html>
- Boehm, B. W. (1981). *Software engineering economics*. Prentice-Hall.
- Conventional Commits. (s.f.). *Conventional Commits 1.0.0*. <https://www.conventionalcommits.org/en/v1.0.0/>
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. IT Revolution Press.
- GitHub. (s.f.). *GitHub Actions documentation*. <https://docs.github.com/actions>
- Google. (s.f.). *Gemini API documentation*. <https://ai.google.dev/gemini-api/docs>
- IEEE. (2014). *IEEE Std 730-2014: IEEE standard for software quality assurance processes*. IEEE.
- International Organization for Standardization & International Electrotechnical Commission. (2009). *ISO/IEC 20926:2009 Software and systems engineering — Software measurement — IFPUG functional size measurement method 2009*. ISO.
- International Organization for Standardization & International Electrotechnical Commission. (2016). *ISO/IEC 25023:2016 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Measurement of system and software product quality*. ISO.
- International Organization for Standardization & International Electrotechnical Commission. (2023). *ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. ISO.
- International Organization for Standardization, International Electrotechnical Commission, & IEEE. (2017). *ISO/IEC/IEEE 12207:2017 Systems and software engineering — Software life cycle processes*. ISO.
- International Organization for Standardization, International Electrotechnical Commission, & IEEE. (2018). *ISO/IEC/IEEE 90003:2018 Software engineering — Guidelines for the application of ISO 9001:2015 to computer software*. ISO.

- International Organization for Standardization, International Electrotechnical Commission, & IEEE. (2021). *ISO/IEC/IEEE 29119-3:2021 Software and systems engineering — Software testing — Part 3: Test documentation*. ISO.
- Jones, C. (2008). *Applied software measurement: Global analysis of productivity and quality* (3.^a ed.). McGraw-Hill.
- McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320.
- McConnell, S. (2004). *Code complete* (2.^a ed.). Microsoft Press.
- Project Management Institute. (2017). *A guide to the project management body of knowledge (PMBOK guide)* (6.^a ed.). PMI.
- Śliwerski, J., Zimmermann, T., & Zeller, A. (2005). When do changes induce fixes? *Proceedings of the 2005 International Workshop on Mining Software Repositories*, 1–5.
- Valle Cortes, I. (2026). *QualityOps AI* [Código fuente]. GitHub.
<https://github.com/TaikoVC/qualityops-ai>